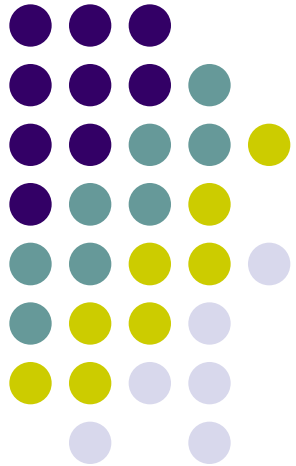
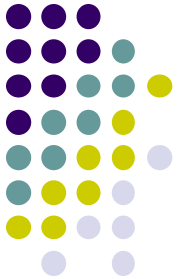
WIX1002
Fundamentals of Programming

Chapter 1
Problem Solving in Programming

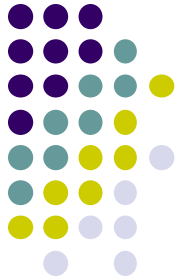




Contents

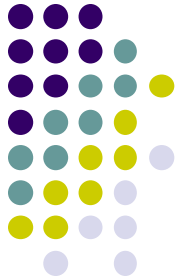
- Introduction
- Java
- Problem Solving
- Input Process Output
- Pseudocode
- Flow Chart
- Sample Java Program

Introduction



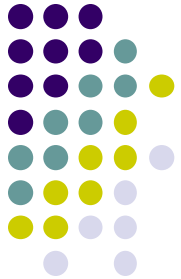
- A computer can be define as an electronic machine, operating under the **control of instructions** stored in its own memory, that can accept data, manipulate that data and produce results that can be stored for future use.
- These set of instructions is called **program**.
- A program contains a large number of operations and a computer must be programmed to perform different tasks.
- A programming language can be divided into **machine language, assembly language and high level language**.

Introduction



- **Machine Language**
  - It is the natural language of a particular computer.
  - It is defined by the hardware design of the computer.
  - It consist of strings of number (**0 and 1**) that construct computers to perform their operations.
  - It is **machine dependent**, particular machine language can be used only for one type of computer.
  - It is cumbersome for humans.

Introduction



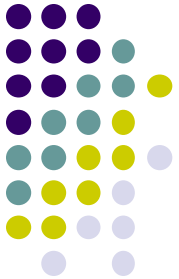
- **Assembly Language**
  - It is the English-like abbreviations languages.
  - Translator programs called **assemblers** were developed to convert assembly language programs to machine language at computer speeds.
  - It is more clearer and more easier to understand as compare to machine language.
  - However, it requires **many statements** in order to perform a simple tasks.
  - Example, **MOV AL, 88h**

Introduction



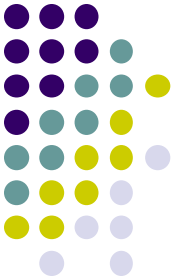
- **High Level Language**
  - It is developed to speed the programming process.
  - It was developed in which single statements could be written to accomplish substantial tasks.
  - **Compiler** is used to convert high level language programs into machine language.
  - High level languages allow programmers to write instructions that look almost like everyday English and contain commonly used mathematical notations.
  - Java is one of the High Level language.

Java

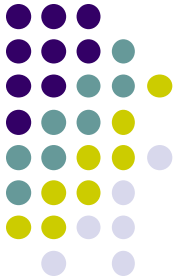


- Java is an **Object-Oriented Programming** language.
- In **OOP**, all the things around us is made up of **objects,** such as people, buildings, vehicles, and etc. Each of the object will has the ability to perform certain **actions** and these actions will has some effect on some other objects in the world. Example a people driving a car.
- Java is a well-know programming language for different applications such as console application, desktop application, distributed application, Internet application and mobile application.
- Besides, Java can be used to develop **Java applet** which can be executed in Web browser or applet viewer.

Java



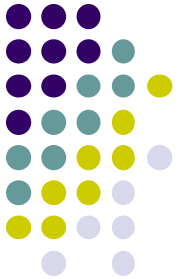
- Java programs normally go through five phases to be executed. These are: **edit, compile, load, verify and execute**.
- **Phase 1 - Edit**
  - This phase consists of editing a file. This is accomplished with an editor program or **Integrated Development Environment (IDE).** Some of the popular IDE includes **NetBeans**, **Eclipse**, Jcreator and JDeveloper.
  - Java program file names end with **.java** extension. The file name must be same as the **Class name**.



Java

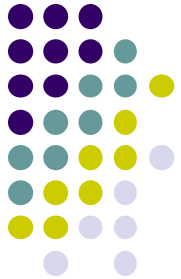
- **Phase 2 – Compile**
  - In this phase, the Java compiler translates the Java program into **byte codes**. The compiler will check whether the source code has the correct spelling and punctuation, all the data types are correct, and all the variables names are legal.
  - Java Platform (**JDK 15**) provides a compiler you can use to compile all kinds of Java programs.
  - Command use to compile java file
    - **javac filename.java**
  - **Syntax** – The **grammar rules** of the language. It tells what arrangement of words and punctuations are allowed in a Class or program definition.

Java



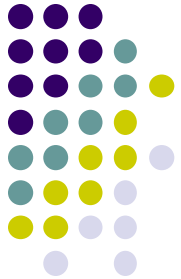
- **Phase 3 – Load**
  - The class loader will take the **.class** file containing the byte codes and transfers it to memory. Java applications are loaded into memory and executed using the Java interpreter.
  - Java interpreter is sometimes referred to as the "Java Virtual Machine" or "Java run-time system".
- **Phase 4 – Verify**
  - In this phase, the byte code verifier is used to verify the byte code before executed. It ensures byte codes do not violate security requirements.

Java

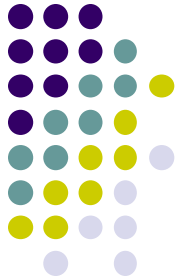


- **Phase 5 – Execute**
  - After verification, the computer will interpret the program one byte code at a time, thus performing the actions specified by the program.
  - Command use to execute java file
    - **java filename**
  - A Java Achieve (JAR) file can be created for a Java application that consists of multiple java files.
  - Command use to execute jar file
    - **java –jar filename.jar**

Problem Solving



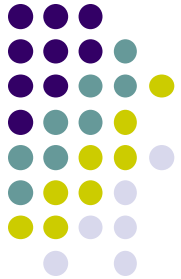
- The purpose of writing a program is to solve a problem or performing certain tasks.
- Before writing a program to solve a problem, it is essential to have a clear understanding of the problem and a carefully planned approach to solve a problem.
- Any computing problem can be solved by executing a series of actions in a **specific order**.
- A procedure for solving a problem in terms of the actions to be executed and the order in which actions are to be executed is called an **algorithm**.



Problem Solving

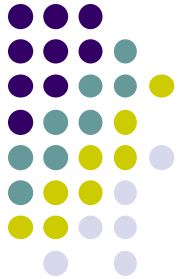
- Below are the general steps of problem solving in programming
  - First, understand the problem. Solve the problem manually with a few examples.
  - Then, devise an algorithm to solve the problem.
  - After that, write the program using the programming syntax and compile the program.
  - Correct the syntax error after the compilation. Save the program and compile again.
  - Execute the program if the program is error free.
  - Finally test the results against expected output. If the results are not correct, modify the algorithm, rewrite and recompile the program

Problem Solving



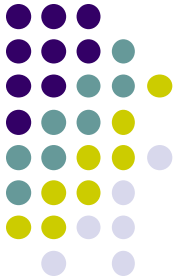
- **Strategies**
  - What do I know about the problem?
  - What is the information that I have to process in order the find the solution?
  - What does the solution look like?
  - What sort of special cases exist?
  - How will I recognize that I have found the solution?

Problem Solving



- **Syntax** – A set of rules, principles, and processes that govern the **structure of statement** in a programming language.
- **Semantic** – Describe the **meaning** of the things written while following the syntax rules of the language. Semantic describes the things happen when a program is executed.
- **Debugging** – A process of **eliminating mistakes** in the program. A mistake in a program is called a bug.

Problem Solving



- There are three commonly types of bugs or errors.
- **Syntax Error**
  - A grammatical mistake in the program. A mistake in the arrangement of words and punctuations.
- **Logic Error**
  - A mistake in the underlying algorithm or semantic error.
- **Run-time Error**
  - An error that happen when the program is executed

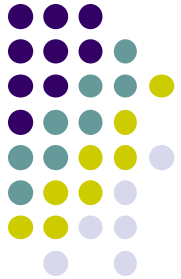
Input Process Output

- A program usually receives inputs from a user or other source (files), does some computations on the inputs (process), and returns the results of the computations (output).

Input	Process	Output
• Number 1 • Number 2 • Add, Subtract, Multiply or Divide	• Assign Variable for Number 1 • Assign Variable for Number 2 • Select Case for Calculation: Add, Subtract, Multiply or Divide • Calculate Number 1 and Number 2	• Result of Calculation

Input	Processing	Output
• Radius of circle	***Processing Items:*** area = π * radius^2 circumference = 2 * π * radius ***Algorithm:*** Step 01: Start Step 02: Input *radius* from the user Step 03: set *PI* = 3.1415 Step 04: Calculate area as: *area = PI * radius^2* Step 05: Calculate circumference as: *circumference = 2 * PI * radius* Step 06: Print *area* and *circumference* Step 07: End	• Area of circle • Circumference of circle

Pseudocode



- Pseudocode is an informal high-level description of the operating principle of a computer program or algorithm. It is simply a numbered list of instructions to perform some task.
- Best Practices
  - Write only one statement per line and each statement should express just one action.
    - Get the name of the participant.
    - Compute the total salary.
    - Display the number of students.

Pseudocode

```
Compute Final Price

1. Get the price of the item
2. Get the sales tax rate
3. Sales Tax = price of item * sales tax rate
4. Final Price = price of item + Sales Tax
```

- Indent statements that fall inside a selection or loop structure.

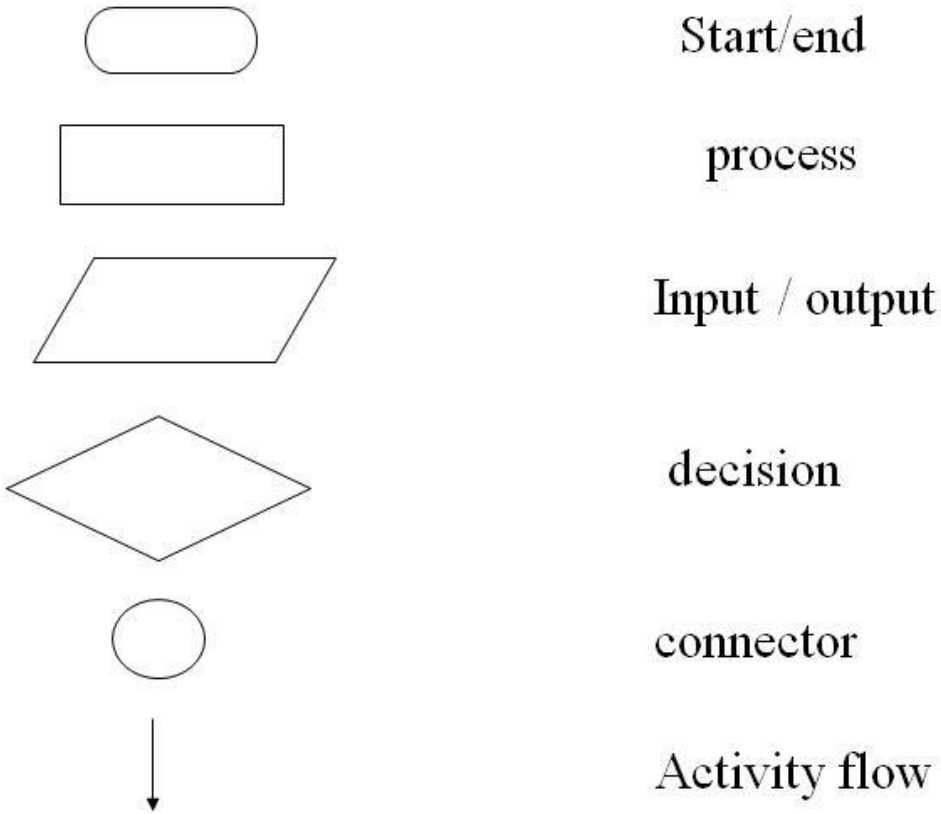
```
If the number of students are more than 40
   Create extra classes
Otherwise
   Create one class
```

```
while the water temperature is greater than 50
   add more ice
   display the new water temperature
```

Flow Chart

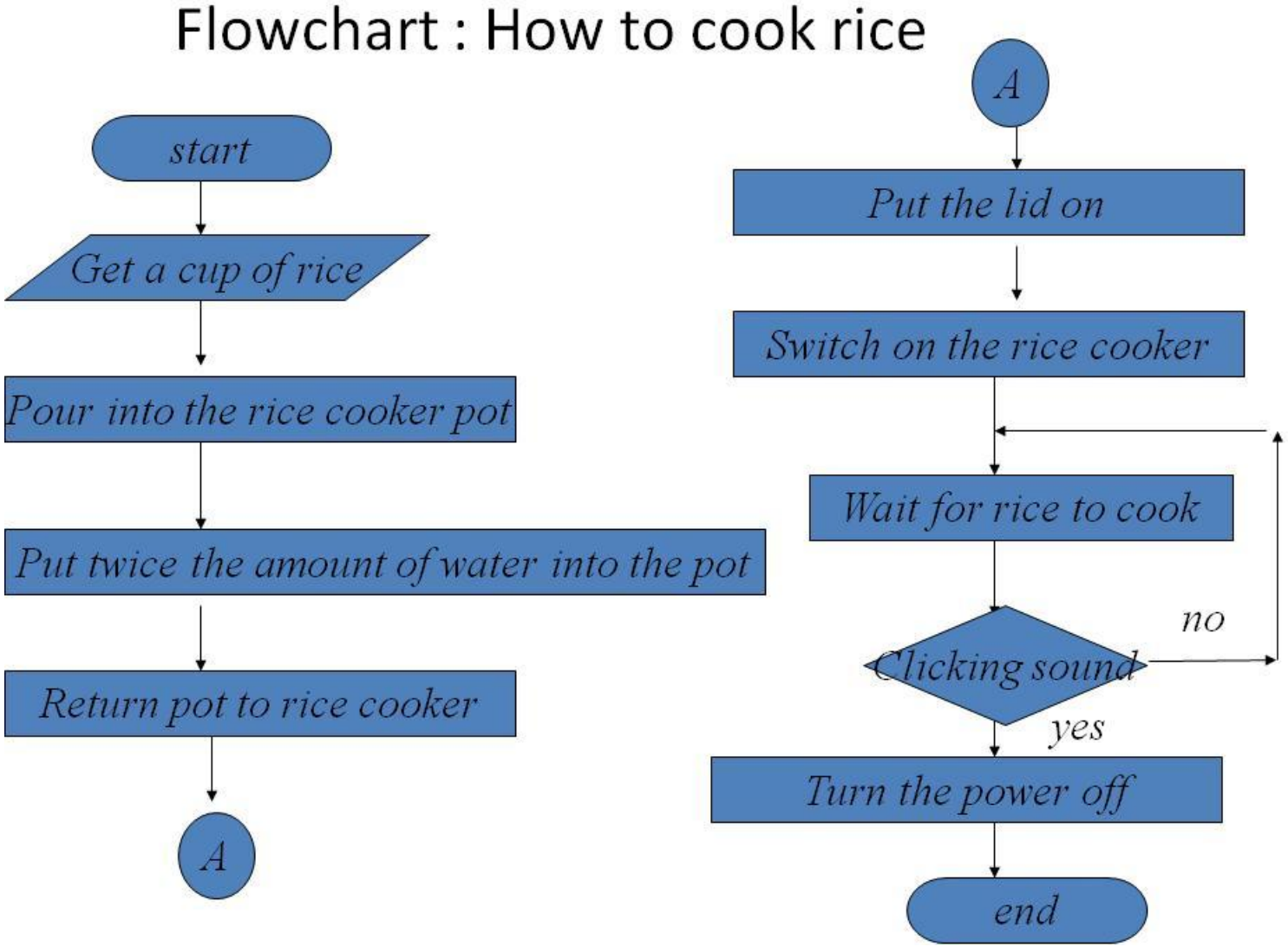
- A flowchart is a type of diagram that represents an algorithm or process.
- A flow chart shows the steps as boxes of various kinds, and their order by connecting them with arrows.
- It is used in analysing, designing, documenting or managing a process or program.

Flow Chart

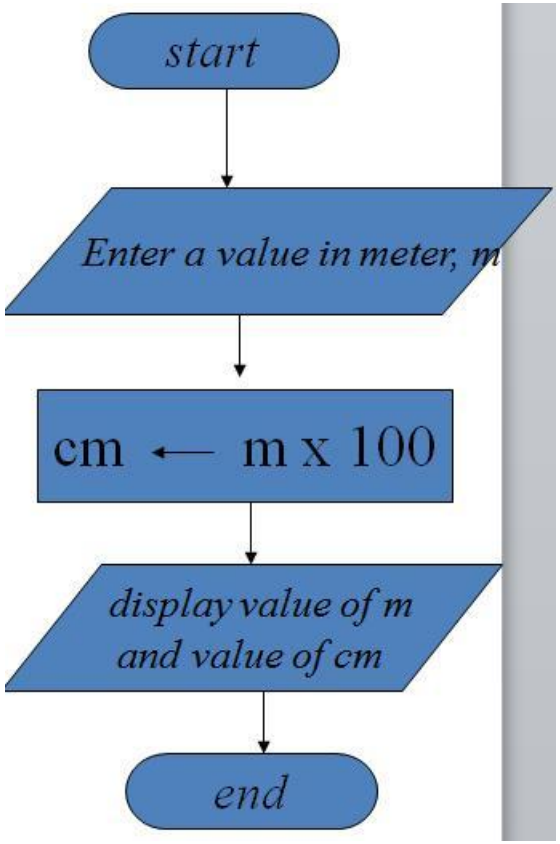


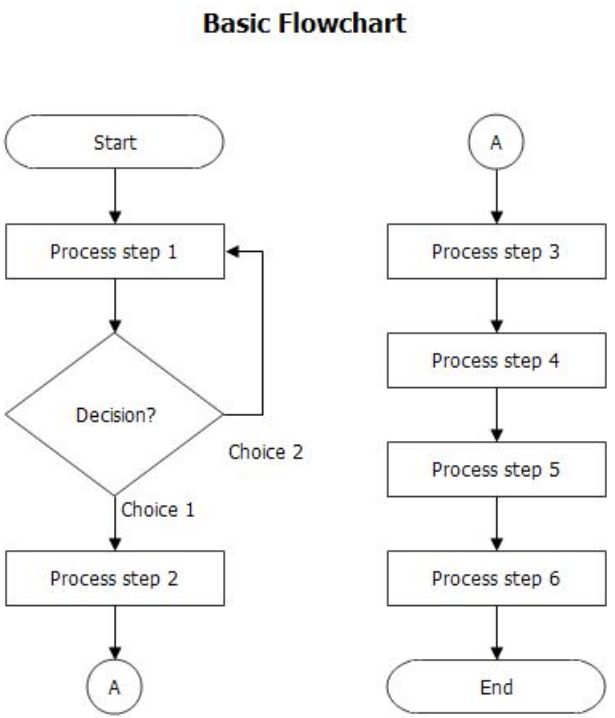
Flow Chart Notation

Flow Chart

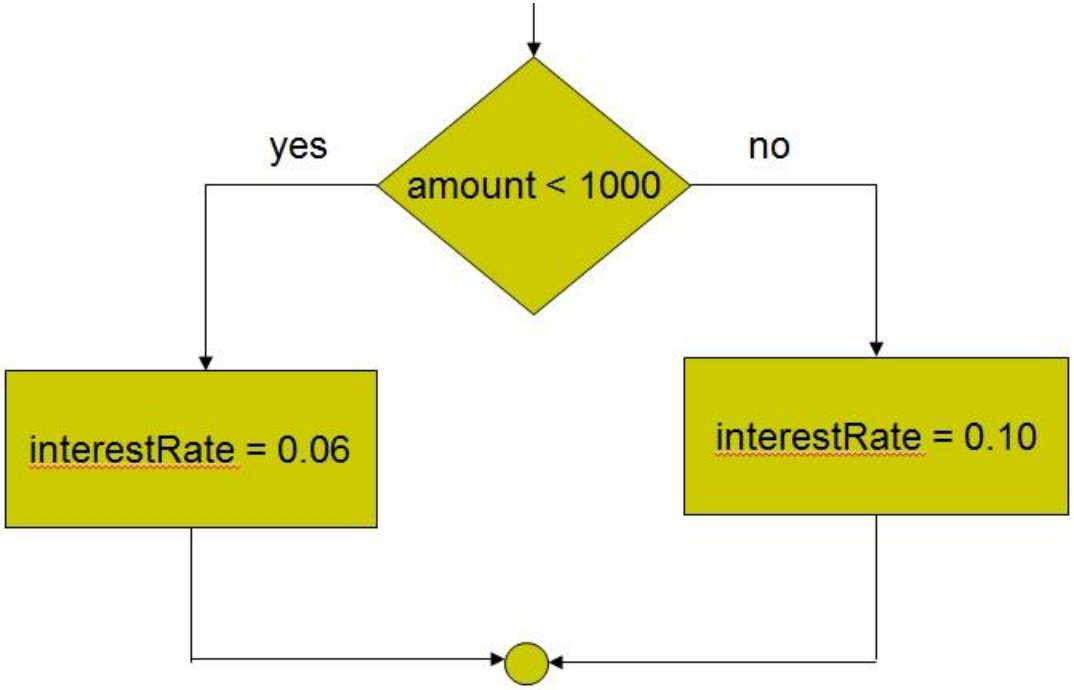

Flowchart : How to cook rice
start
Get a cup of rice
Pour into the rice cooker pot
Put twice the amount of water into the pot
Return pot to rice cooker
A
A
Put the lid on
Switch on the rice cooker
Wait for rice to cook
Clicking sound
no
yes
Turn the power off
end

Flow Chart


start
Enter a value in meter, m
cm ← m x 100
display value of m
and value of cm
end

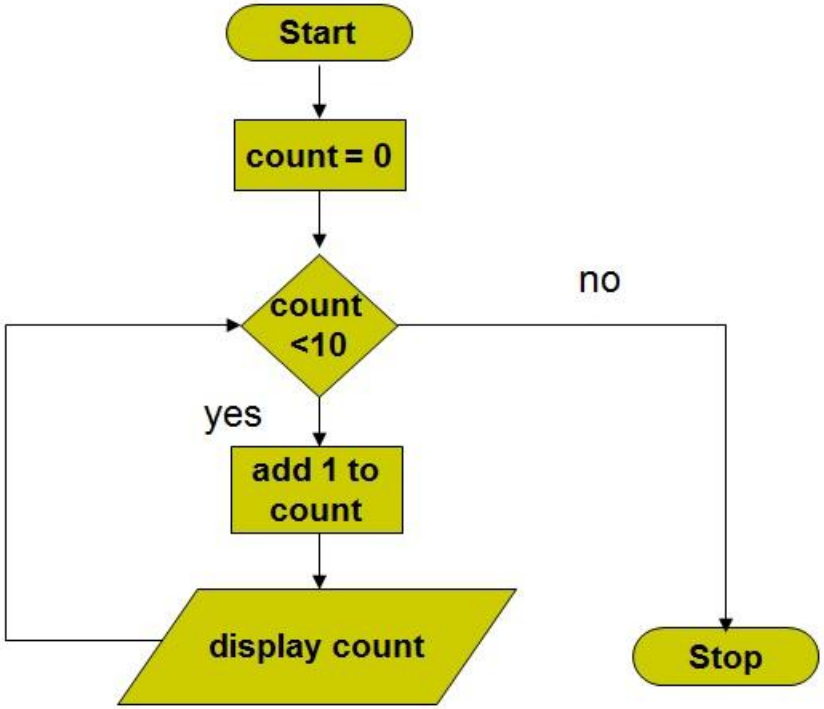

Basic Flowchart
Start
Process step 1
Decision?
Choice 2
Choice 1
Process step 2
A
A
Process step 3
Process step 4
Process step 5
Process step 6
End

Flow Chart



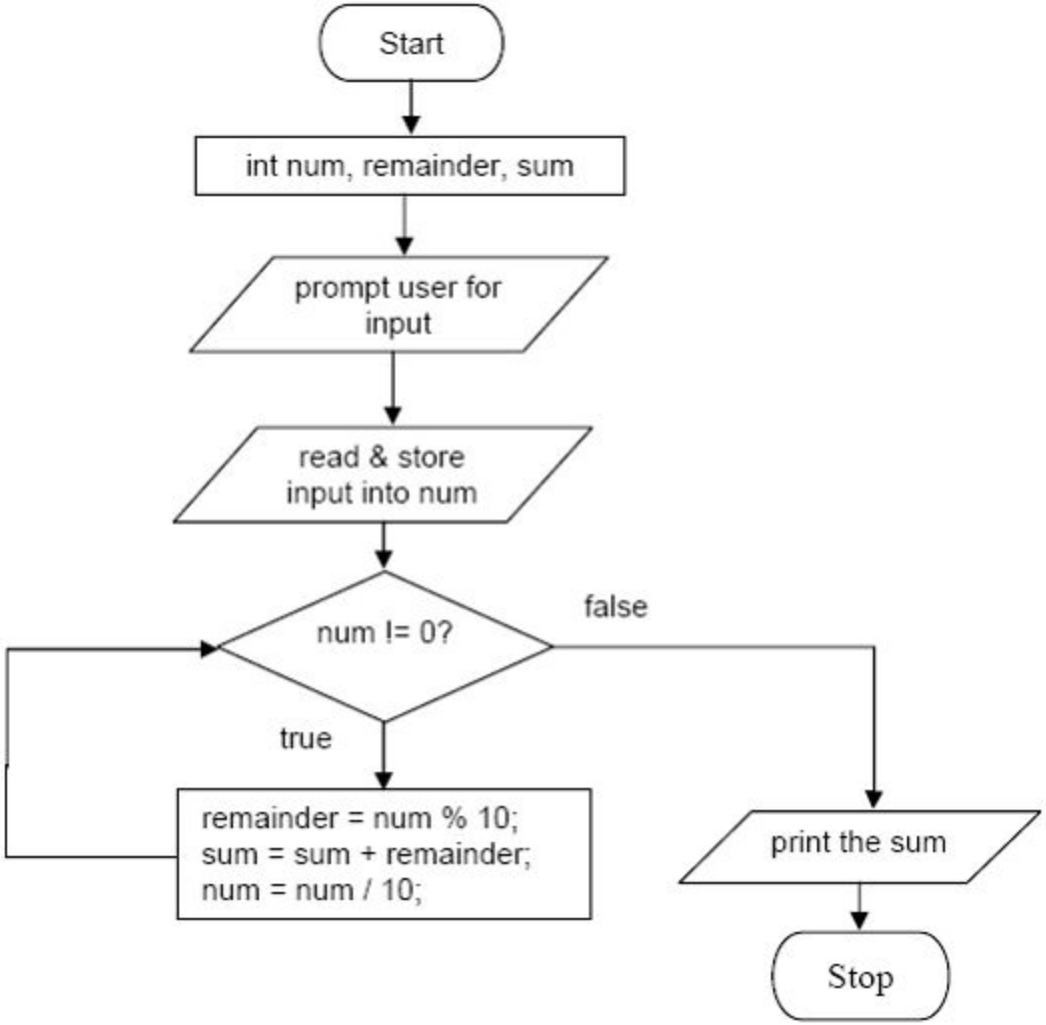
Selection Structure

Flow Chart



Repetition Structure

Flow Chart

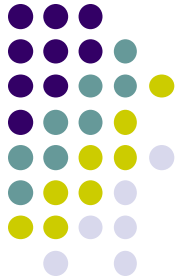

Start
int num, remainder, sum
prompt user for input
read & store input into num
num != 0?
false
true
remainder = num % 10;
sum = sum + remainder;
num = num / 10;
print the sum
Stop

Sample Java Program

```
public class FirstProgram
{
   public static void main(String[] args)
   {
      System.out.println("Welcome to Java!");
   }
}
```

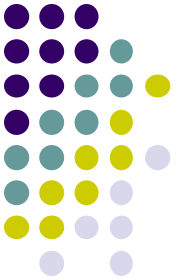
* File Name: **FirstProgram.java**

Sample Java Program

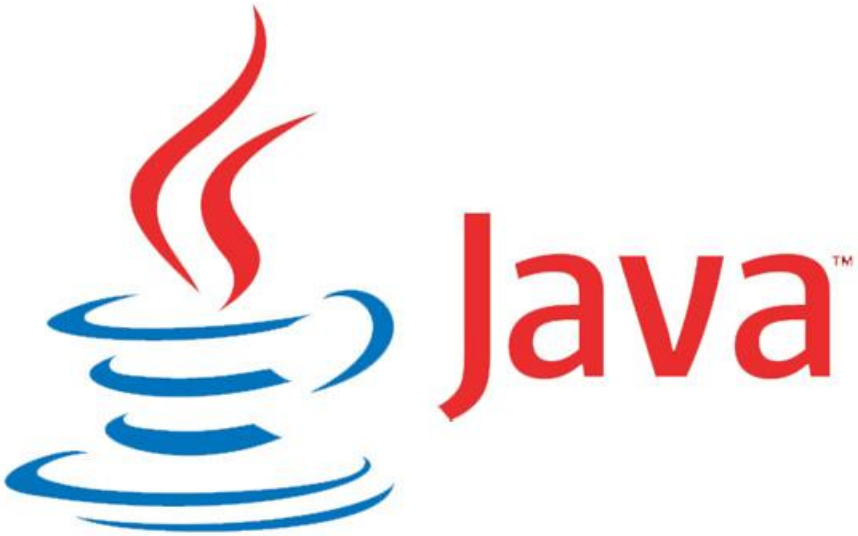


- Every program in Java consists of **at least one class definition.** The class name of the program is **FirstProgram**.
- A left brace { - begins the body of every class definition. A right brace } - ends each class definition.
- The class contains a method named **main**. When the program is run, the **main** method is invoked which means the statements in the **main** method are executed.
- The **void** indicates that this method will perform a task and will not return any information when the task is completed.
- The statement in the sample program display the output **Welcome to Java!**

Sample Java Program

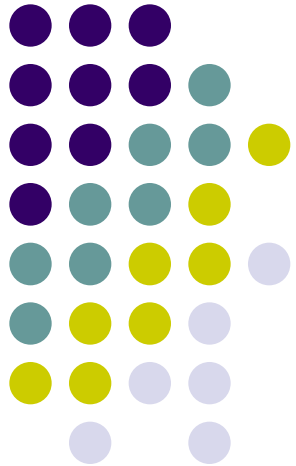


- In Java:
  - A program is made up of one or more **classes**
  - A class contains one or more **methods**
  - A method contains program **statements**
- Java is **case sensitive**. The uppercase letters and lowercase letters must be enter correctly.
- Class are the fundamental building blocks of Java program. Each program begin with a **class definition**.
- Every **Java application** contains a **main method**. The instructions in the main method are executed when the application starts. main method must always be **static**.
- Each Java statements must end with a semicolon (;)


Java™

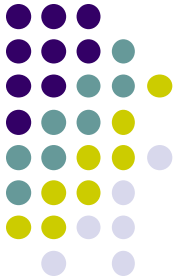
WIX1002
Fundamentals of Programming

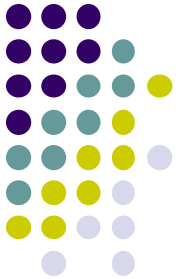
Chapter 2
Java Fundamental



Contents

- Variable
- Operator
- Type Casting
- String
- Console Input
- Console Output
- Comment
- Random Number





Variable

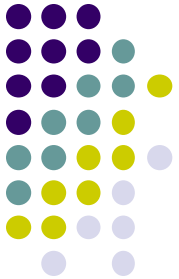
- A variable is a storage location in the memory that has a **type, name and contents**.
- A variable must be **declared**, specifying the variable's **name** and the **type** of information that will be held in it.
- **Identifier** is the name of the variable. In Java, identifier may contains letters, digits (0 – 9), and the underscore (_) character.
- **However, identifier cannot begin with a digit, cannot contain spaces and cannot be reserved words.**
- A meaningful identifier will made the program easy to understand.
- By convention, variable names should start with a lowercase letter

Variable

- Java has basic types for characters, integers and floating points number. These basic types are known as **primitive types**.
- Java Unicode character chart
  - http://www.ssec.wisc.edu/~tomw/java/unicode.html

Java Primitive Data Types				
Type	**Values**	**Default**	**Size**	**Range**
byte	signed integers	0	8 bits	-128 to 127
short	signed integers	0	16 bits	-32768 to 32767
int	signed integers	0	32 bits	-2147483648 to 2147483647
long	signed integers	0	64 bits	-9223372036854775808 to 9223372036854775807
float	IEEE 754 floating point	0.0	32 bits	+/-1.4E-45 to +/-3.4028235E+38, +/-infinity, +/-0, NAN
double	IEEE 754 floating point	0.0	64 bits	+/-4.9E-324 to +/-1.7976931348623157E+308, +/-infinity, +/-0, NaN
char	Unicode character	\u0000	16 bits	\u0000 to \uFFFF
boolean	`true, false`	`false`	1 bit used in 32 bit integer	NA

Variable



- **declaration**

  int number;

  double first, second;

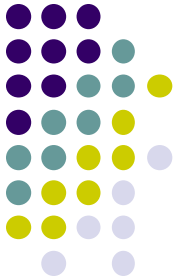
  boolean status;

- **initialization**

  double speed = 2.6;

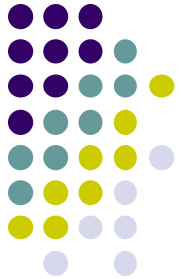
  char letter = 'A'; //use single quote for character

Variable



- A **constant** is an identifier that is similar to a variable except that it holds one value for its entire existence
- The compiler will issue an error if you try to change a constant variable
- Constant is declared using the **final** keyword.
  - final int MIN=0;
  - final int MAX=100;

Operator



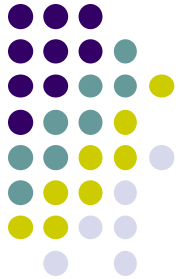
- Operators are special symbols that perform specific operations on one or more **operands**.
- An operand is the part of a computer instruction which specifies what data is to be manipulated or operated on.
- **Assignment Operator (=)**
  - It is used to change the value of variable after the variable has been declared.

    temperature = 36.9;

    grade = 'c';

    status = true;

Operator



- **Arithmetic Operator**
  - It is used to compute numeric results
  - + (addition) - (subtraction) * (multiplication) / (divison) % (modulo or remainder)

    temperature = a + 36.9;

    total = monthlyPayment * 12;

    answer = 20 % 6; // answer is 2

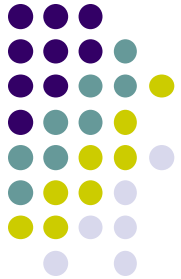
Operator

- Java follows **precedence rules** that determine how the operators are executed in order

Operator Precedence

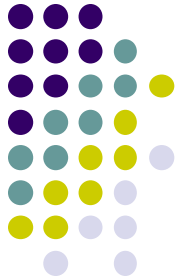
Operators	Precedence
postfix	*expr*++ *expr*--
unary	++*expr* --*expr* +*expr* -*expr* ~ !
multiplicative	* / %
additive	+ -
shift	<< >> >>>
relational	< > <= >= instanceof
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	|
logical AND	&&
logical OR	||
ternary	? :
assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Operator



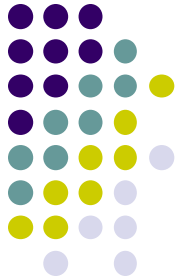
- **Parentheses Operator ( )**
  - Controls the order in which the operators execute in the expression. Parentheses override the normal precedence order
- **Postfix Operator**
  - number++;
    - **Use the current value of number**. Then increment by 1 for the next statement.
  - number--;
    - **Use the current value of number**. Then decrement by 1 for the next statement

Operator



- **Unary Operator**
  - ++number;
    - **Increment number by 1** and then use the value
  - --number;
    - **Decrement number by 1** and then use the value.
- Other assignment operator
  - number += 5; // -=, *=, /=, %=
  - number = number + 5;

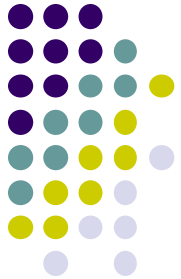
Type Casting



- Sometimes it is convenient to convert data from one type to another. For example, we may want to treat an integer as a double value during a computation
- Type casting takes a value of one type and produces a value of another type.

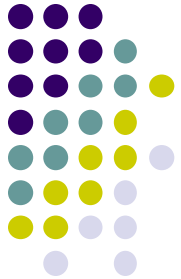
```
int a=8, b=3;
double answer;
answer = a / b;                 // Output is 2
answer = a / (double) b;        // Output is 2.6666
```

String



- A string is a sequence of characters that is treated as a single value.
- String class is used to store and process **strings of characters**.
- Declaration and initialization
  - String fullname;
  - String topic = "Object-oriented Programming";
- Concatenation
  - + operator is used to connecting two strings.
  - String firstName, lastName, fullname;
  - fullname = firstname + lastname;

Console Input

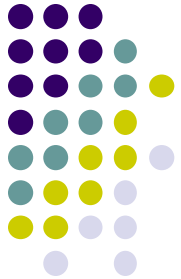


- **Scanner Class**
  - A simple text scanner which can used to **get input for primitive types and strings**.
  - To load Scanner Class into java program

    import java.util.Scanner;

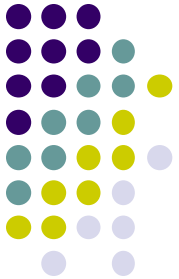
```
Scanner keyboard = new Scanner(System.in);
int num;
System.out.println("Please enter a number");
num = keyboard.nextInt();
        System.out.println(“The number is " + num);
```

Console Input



- .nextInt()
  - Reads one int from the keyboard
- .nextLong()
  - Reads one long int from the keyboard
- .nextDouble()
  - Reads one double from the keyboard
- .next()
  - Reads **one word** into String class from the keyboard
- .nextLine()
  - Reads an entire line into String class from the keyboard

Console Output



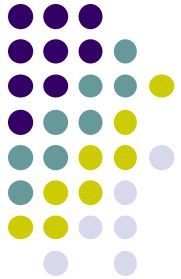
- System.out is known as the standard output object
- **System.out.println** display a line of text in the command window. When it completes its tasks it automatically positions the output cursor to the beginning of the next line.
- **System.out.print** display a line of text in the command window. However, it does not position the output cursor at the beginning of the next line in command window.
- **System.out.printf** display output in a **specific format**

Console Output

Code	Output	Examples
d	decimal integer	%d %6d
f	fixed-point floating point	%f %6.2f
e	E-notation floating point	%e %4.2e
g	general floating point	%g %4.2g
s	string	%s %20s
c	character	%c %5c
-	left alignment	%-5d

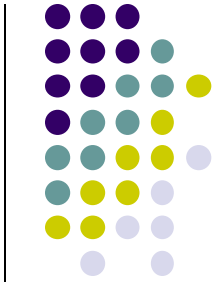
- System.out.printf("%6.2f", 22/7.0)
  - Display 6 spaces with 2 decimal places as in _ _3.14
- System.out.printf("PI%-6.2fValue", 22/7.0)
  - Display PI3.14_ _Value

Console Output



- You can use some of the special characters in the standard output object
- Special Character
  - **\n** is the **newline character**. It cause the screen's output cursor to move to the beginning of the next line
  - **\t** is the **horizontal tab** character
  - \\ display a backslash character
  - **\"** display a double quote character

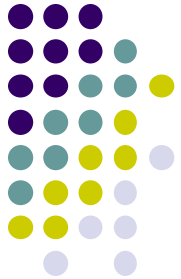
Comment



- Comment is used to help other programmers to understand the program
- Comments **are not executed** when the application start.
- There are three types of comment in Java
- Single line comment
  - *//* A Single Line Comment
- Multiple lines comment

```
/*
    A multiple
    line comments
*/
```

Comment



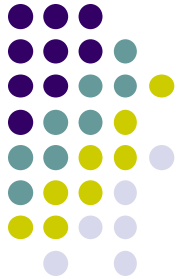
- Documentation comment describe the functionalities of each Java class and methods.

  /**

  This method display a line of text on the screen

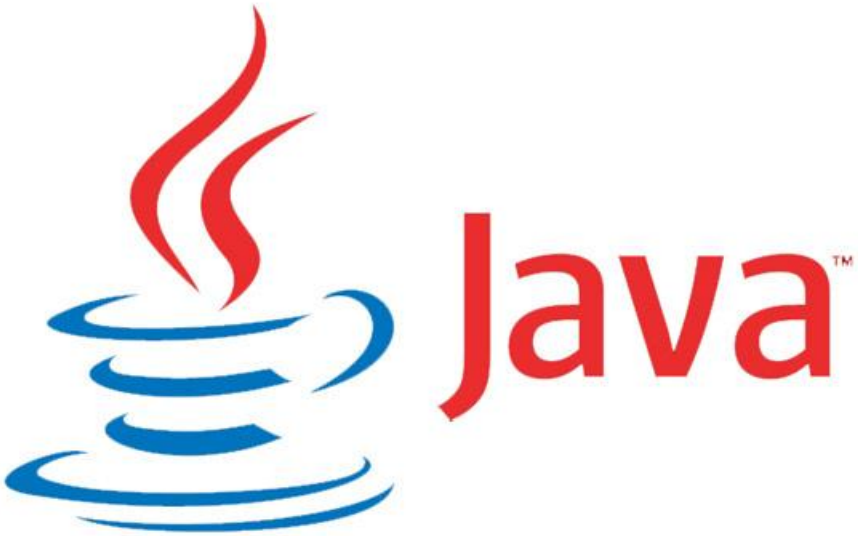
  ****/***
- How to write a standard Java Documentation Comment
  - https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html
- To run javadoc on a package
  - **javadoc -d documentationDirectory PackageName**

Random Number



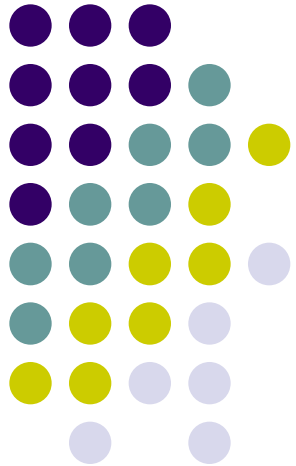
- Random number is very useful in game and simulation
- The Random class of Java implements a random number generator
  - **import java.util.Random;**

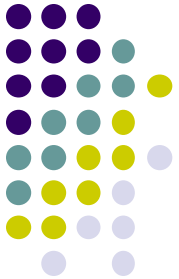
```
Random g = new Random();
int num;
num = g.nextInt(); //any random value of integer
int MAX = 100;
num = g.nextInt(MAX); // random value from 0 to 99
```


Java™

WIX1002
Fundamentals of Programming

Chapter 3
Flow of Control (Selection)

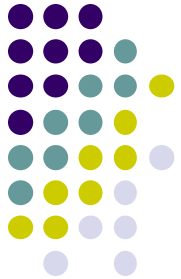




Contents

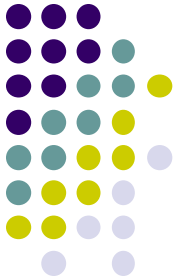
- Introduction
- Relational Operator
- Logical Operator
- if
- if-else
- ultiway if-else
- switch
- Ternary Operator
- Common Error

Introduction



- Flow control is to used to specify the order of the statements to be executed.
- Program can be written in three types of flow control namely the **sequence, selection and repetition**.
- When the statements are executed **one after the other** in order, it is called the **sequence flow**.
- A **selection flow** chooses among **alternative courses of action**.
- A **repetition flow** specifies that an action is to be **repeated while some condition remains true**.

Introduction



- Computer programs often need to make decisions, taking different actions depending on the condition.
- In Java, **if** and **switch** statement are used to carry out the decision.
- The statement is controlled by the **boolean expression**.
- **Relational/Conditional operator** can be used in the boolean expression.
- If there is more than one **constraint/condition** in the decision making, the **logical operators** are used to merge multiple constraint/conditions.

Relational Operator

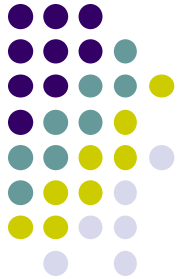
- The relational operator tests the relationship between two values.

Operator	Description	Examples
==	Equal	a==b
!=	Not Equal	c!=d
>	Greater than	x>y
>=	Greater than or equal	x>=y
<	Less than	a<b
<=	Less than or equal	a<=b

Logical Operator

- The logical operator is used to create complex Boolean expression by merging multiple constraints/conditions.

Operator	Description	Examples
&&	AND (true && true is true, others false)	a==b && c==d
||	OR (false && false is false, others true)	c!=d || a<b
!	NOT (!false is true. !true is false)	!(x>y)



if

- if statement is used to implement a decision. It consists of condition and body.
- If the condition is true, the body of the statement is executed.

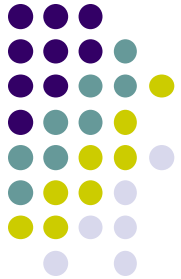
```
if ( condition1 )
    statement 1;

// use brace {                    more than 1 statements
if ( condition1 ) {
    statement 1;
    statement 2;
}
```

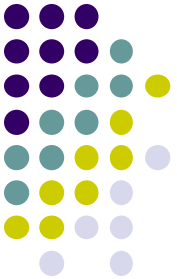
if

```
if (number > 0)
    System.out.println("The number is positive");

if (result < 50) {
    System.out.println("You did not pass");
    System.out.println("Try harder next time");
}
```



if-else



- if-else statement chooses between two alternative statements based on the condition or boolean expression

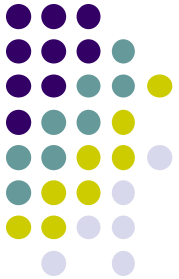
```
if ( condition1 )
    statement 1;
else
    statement 2;


if (myScore > yourScore)
    System.out.println("I Win!");
else
    System.out.println("You Win!");
```

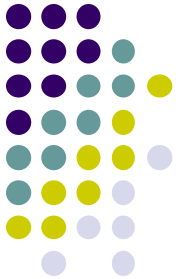
if-else

```
// use brace {
if ( condition1 ) {
    statement 1;
    statement 2;
}
else {
    statement 3;
    statement 4;
}
```

more than 1 statements



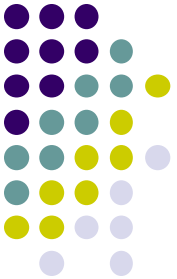
if-else



- String Comparison
  - When testing string for equality, **DO NOT USE ==** operator. Use **equals** or **equalsIgnoreCase**.
  - String.equals(other_string)
  - String.equalsIgnoreCase(other_string)

```
String s1, s2;
if (s1.equals(s2))
   System.out.println("They are equal strings.");
else
   System.out.println("They are not equal strings.");
```

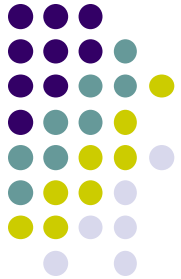
if-else



- **Alphabetical Order**
  - Lexicographic ordering is used to order alphabet according to ASCII ordering. Use **compareTo** and **compareToIgnoreCase**.
  - String.compareTo (other_string)
  - String.compareToIgnoreCase(other_string)

  - s1.compareTo(s2)
    - Return **negative** value if **s1 comes before s2**.
    - Return **positive** value if **s2 comes before s1**.
    - Return zero if s1 is equal to s2.

Multiway if-else

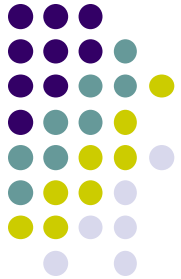


- Multiway if-else statement is the if-else statement nested inside the if-else statement

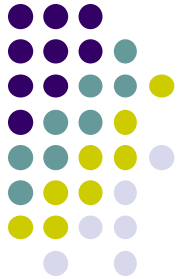
```
if ( condition1 )
    statement 1;
else if ( condition2 )
    statement 2;
else if ( condition3 )
    statement 3;
else
    statement 4;
```

Multiway if-else

```
if (myScore > yourScore) {
    System.out.println("I Win!");
}
else if (myScore < yourScore) {
    System.out.println("You Win!");
}
else {
    System.out.println("Tie!");
}
```



switch



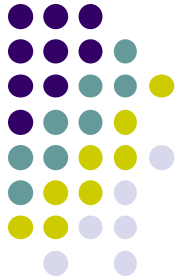
- switch statement can be used to represent multiway if-else statement.

```
switch ( variable ) {
case value1:
    statement 1;
    break;
case value2:
    statement 2;
    break;
default:
    statement 3;
}
```

switch

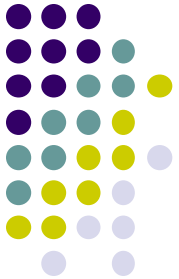
```
switch (number) {
   case 1:
      System.out.println("Satu");
      break;
   case 2:
      System.out.println("Dua");
      break;
   case 3:
      System.out.println("Tiga");
      break;
   default:
      System.out.println("This program accepts the number from 1 to 3
     only");
}
```

Ternary Operator



- The ternary operator ? : is similar to if-else statement
- **condition1 ? statement1 : statement2**
  - If the condition1 is true, the statement1 will be executed.
  - If the condition1 is false, the statement2 will be executed.
  - y = x >=0 ? x : -x;

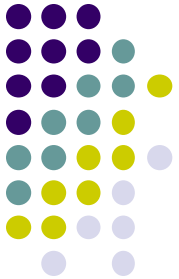
Common Error

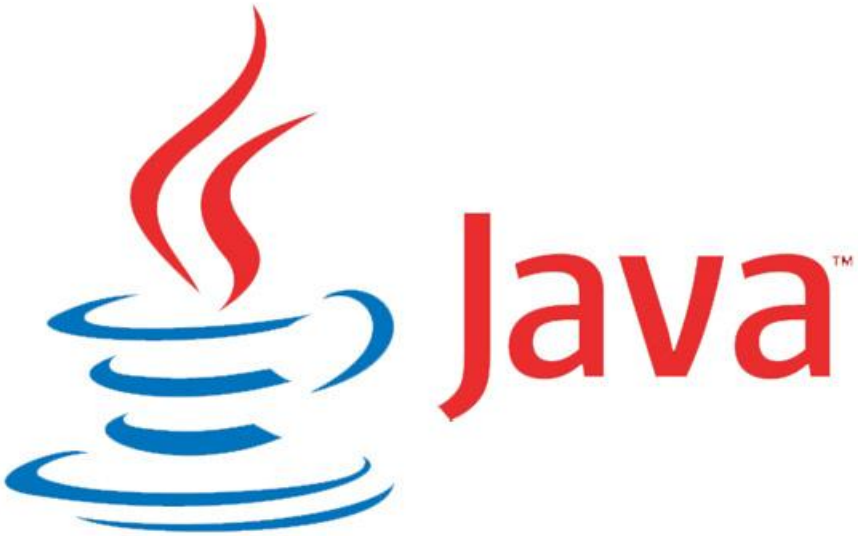


- if number > 0
  - System.out.println("No bracket");
- if (x > 5 * (y - z)
  - System.out.println("Miss one bracket");
- if (0 <= mark <= 100)
  - System.out.println("Invalid syntax");
- if (choice **=** 'Q')
  - System.out.println("Equal sign error");
- if (age > 21 and mark > 80)
  - System.out.println("and should be &&");

Common Error

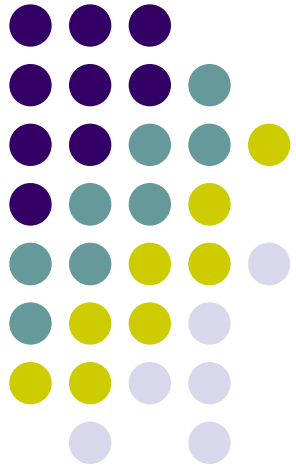
- switch (monthNum) {
  case 1:
  System.out.println("January");
  case 2:
  System.out.println("No break");
  }

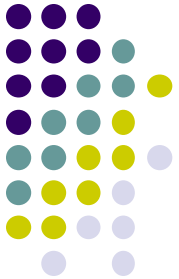



Java™

WIX1002
Fundamentals of Programming

Chapter 4
Flow of Control (Repetition)

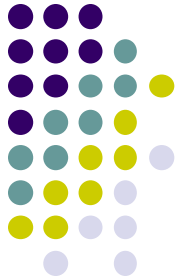




Contents

- Introduction
- while
- do-while
- for
- break
- continue
- label
- Common Error

Introduction



- A repetition flow specifies that an action is to be repeated while some condition remains true.
- In Java, **while, do-while and for** statement are used for the repetition flow.
- There are two types of loop namely count-controlled loop and sentinel-controlled loop.
- **Count-controlled loop** executed the statements for a fixed number of times.
- **Sentinel-controlled loop** executed the statements repeatedly until the sentinel is encountered.

while

- A while statement executes a block of code repeatedly. A **condition** controls how often the loop is executed.

```
while (condition)
        statement;

// use brace {              more than 1 statements
while (condition) {
        statement1;
        statement2;
        statement3;
}
```

while

```
int number=1, sum=0;
while (number<=10) {
   sum+=number;
   number++;
}

boolean status = true;
while(status) {
   number = k.nextInt();
   if (number < 0 )
      status = false;
}
```

do-while

- A do-while statement executes the body of the loop **at least once** and perform condition check after the body statements have been executed.

```
do
    statement;
while (condition);

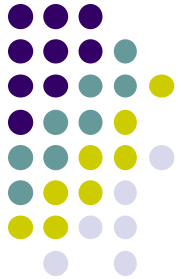
// use brace {          more than 1 statements
do {
    statement1;
    statement2;
} while (condition);
```

do-while

```
int number=1, sum=0;
do {
   sum+=number;
   number++;
} while (number<=10);

boolean status = true;
do {
   number = k.nextInt();
   if (number > 0 )
      status = false;
} while(status);
```

for



- A for statement is suitable for **count-controlled** loops. It is used to step through some integer variable in equal increments or decrements

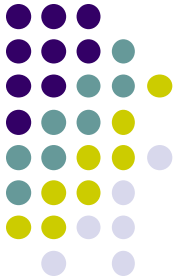
```
for (initialization; condition; update)
        statement;

// use brace {          more than 1 statements
for (initialization; condition; update) {
        statement1;
        statement2;
        statement3;
}
```

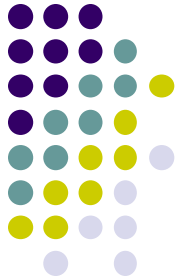
for

```
for (int num = 1; num <= 5; num++)
   System.out.println("Counter is " + num);

for (int i=10; i>0; i--) {
   sum +=i;
   counter++;
}
```



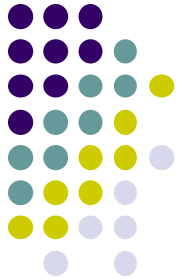
break



- A break statement **ends the nearest enclosing loop** statement

```
for (count = 1; count <= 10; count++) {
   if ( count == 5 )
      break;  // break the loop when count is equal to 5
   sum += count;
}
```

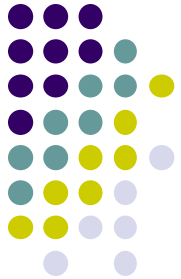
continue



- A continue statement **ends the current loop body iteration** of the nearest enclosing loop statement and proceeds with the next iteration of the loop

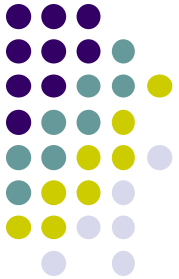
```
for (int count = 1; count <= 10; count++ ) {
   if ( count == 5 )
      continue;  // skip remaining statement in the loop
           // when count = 5
   sum+=count;
}
```

label



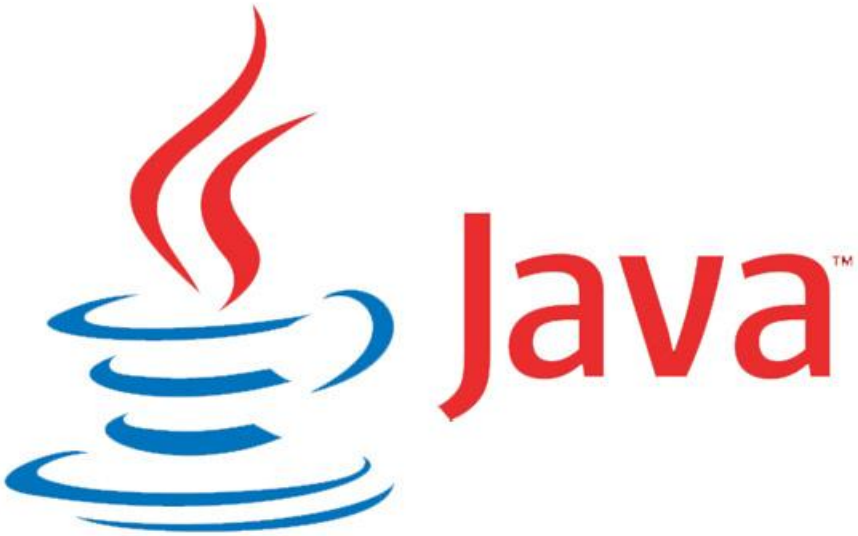
- A label statement is used to **label a loop statement.** The label statement can be used by the break statement and the continue statement

```
stop: {   // label statement
for (int row = 1; row <= 10; row++) {
   for (int column = 1;column <= 5; column++) {
      if ( row == 5 )
         break stop; // break the stop label statement
      counter++;
   }
}
```



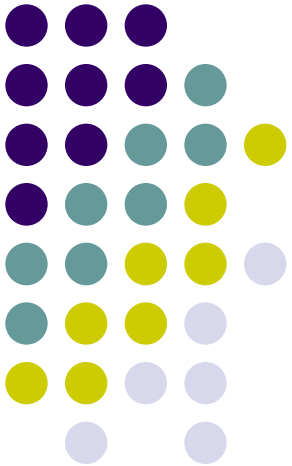
Common Error

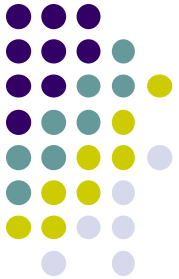
- **An off by one error**
  - The loop iterates once too often or once too few times. Check the condition and / or the initial value of the counter.
- **An infinite counting loop**
  - The counter is going the wrong way or doesn't change at all. Make sure the counter change correctly in loop
- **An infinite sentinel loop**
  - The new data are not input at the end of the loop body


Java™

WIX1002
Fundamentals of Programming

Chapter 5
Arrays

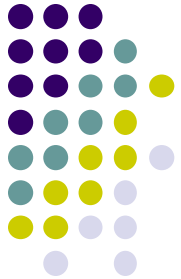




Contents

- Introduction
- Multidimensional Arrays
- Applications
- Bubble Sort
- Linear Search
- Binary Search

Introduction

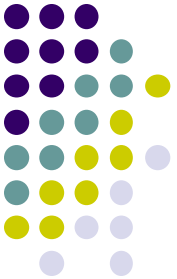


- An array is a **sequence** of values of the **same type**.
- It is a data structure that used to process **a collection of data** that is from the same type.
- An array is a group of **contiguous memory locations** that all have the **same name and the same type**. It is an ordered list of values and each value has a **numeric index** (subscript)
- An array of size N is indexed from **zero to N-1**
- In Java, an array is a **reference data type**.
- The **new** operator is used to construct the array.

Introduction



Introduction

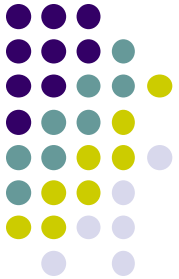


- **Array Declaration**

  typeofArray[] nameOfArray = new typeofArray[length];

  char[] line = new char[100];

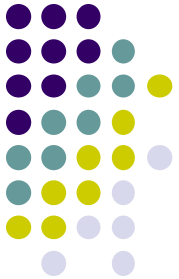
  double[] score = new double[20];
- When declare an array, each element of the array receives **a default value zero for the numeric data type, false for boolean and null for references or String**.
- **Array Initialisation**
  - int[] number = {1,2,3,4,5,6};
  - String[] name = {"one", "two", "three"};



Introduction

- A particular value in an array is referenced using the array name followed by the index (subscript) in brackets.
- The first element in every array start with 0.
  - arrayName[0] refer to the first element
  - arrayName[3] refers to the 4th value in the array.
- Every array in Java knows its own length.
  - **arrayName.length** - determine the length of the array
  - Java will throw an exception if an array index is **out of bounds**.
- Array elements are not limited to primitive data type but also array of object.

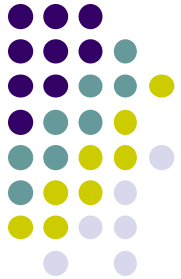
Introduction



- To display the element of the array
  - for(int i=0; i<arrayName.length; i++)
  - System.out.println(arrayName[i]);

- Or
  - for(int value : arrayName)
  - System.out.println(value);
  - However, this method can't be used to modify the value of the element of the array.

Multidimensional Arrays



- We can declare arrays with more than one index in Java.
- **Multidimensional Arrays Declaration**

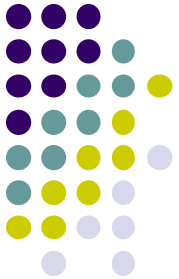
  typeofArray[] … [] nameOfArray = new
  typeofArray[length_i] … [length_n];

  double [][] scoreTable = new double[4][5];

  //4 rows 5 columns
- Multidimensional Arrays with two subscripts are often used to represent tables of values consisting of information arranged in rows and columns.
- **Multidimensional Arrays Initialisation**

  int[][] matrix = { { 1, 2}, { 4, 5} };

Multidimensional Arrays



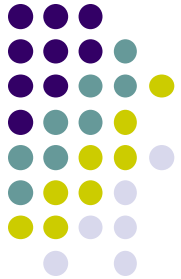
- **Ragged Arrays**
  - Different rows of the array consists of different columns

    int [][]num = new int[3][];

    t[0] = new int[3];

    t[1] = new int[10];

    t[2] = new int[5];



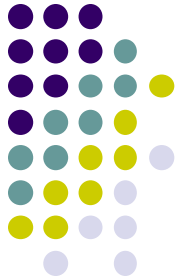
Applications

- Summing elements of an array

```
int a[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
for ( int i = 0; i < a.length; i++ )
  total += a[ i ];
```

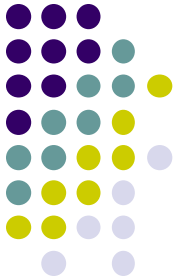
- Sorting is a technique to arrange elements in same order. Sorting data is one of the common feature that can apply on arrays. The common sorting technique is **Bubble Sort**.
- Searching is a technique to locate an item given its key. Searching locating a particular element value in the array. The common searching techniques are **Linear Search** and **Binary Search**.

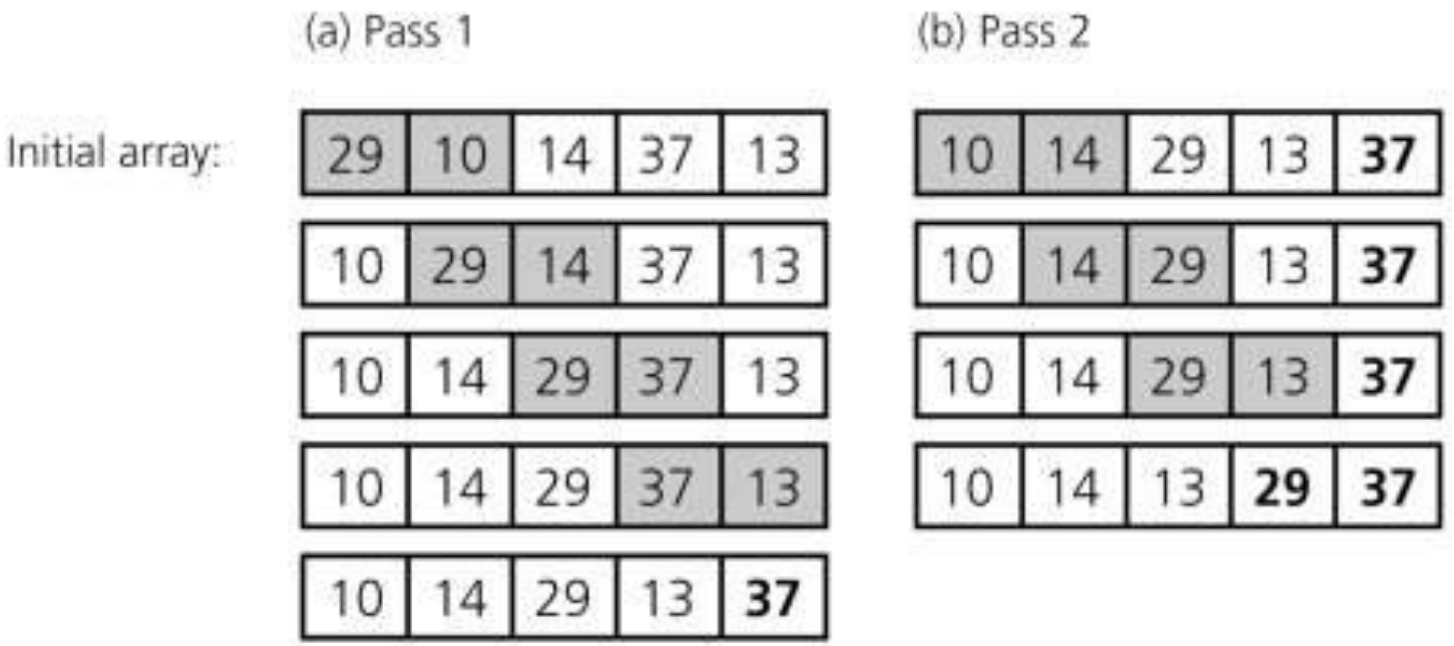
BubbleSort



- The bubble sort uses **nested loops** to make several passes through the array. Each pass compares successive pairs of elements.
- If a pair is in increasing order, the bubble sort leaves the value as they are.
- If a pair is in decreasing order, the bubble sort swaps their values in the array.
- It requires several pass over the data.

BubbleSort

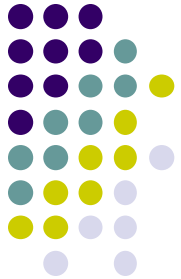



(a) Pass 1
(b) Pass 2
Initial array:
29 10 14 37 13
10 29 14 37 13
10 14 29 37 13
10 14 29 37 13
10 14 29 13 37
10 14 29 13 37
10 14 29 13 37
10 14 13 29 37
10 14 29 13 37

BubbleSort

```
// control number of passes
for ( int pass = 1; pass < b.length; pass++ )
      // control number of comparison
 for ( int i = 0; i < b.length - 1; i++ )
      if ( b[ i ] > b[ i + 1 ] )  {
            int hold = b[i];
            b[i] = b[i+1];
            b[i+1] = hold;
         }
```

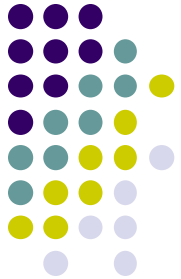
Linear Search



- This method works well for **small arrays or for unsorted arrays**.
- The search technique search the array from the **first position to the last position** in a linear progression.
- It compares each element of the array with a search key.

```
for ( int cnt = 0; cnt < arrayName.length; cnt++ )
    if ( arrayName[ cnt ] == searchKey )
        return cnt;
return -1; // key not found
```

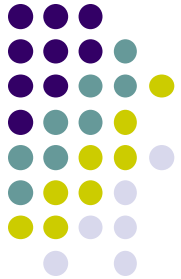
Binary Search



- This method works well for **large and sorted arrays**.
- It eliminates half of the elements in the array being searched after each comparison.
- It locates the middle array element and compares it to the search key. If the search key not found, the binary search reduces the problem to **search for half** of the array.

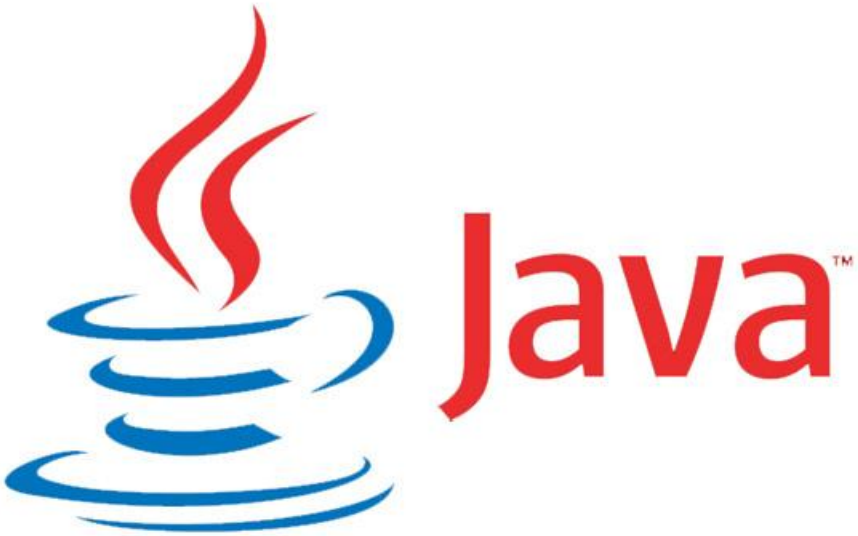
```
int low = 0;   // low element subscript
int high = arrayName.length - 1;  // high element subscript
int middle;    // middle element subscript
```

Binary Search



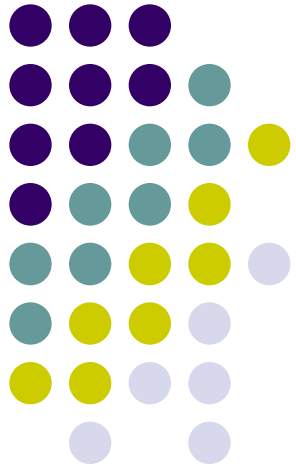
// loop until low subscript is greater than high subscript

```
while ( low <= high ) {
middle = ( low + high ) / 2;
if ( key == arrayName[ middle ] )
        return middle;
else if ( key < arrayName[ middle ] )
        high = middle - 1;
else
        low = middle + 1;
}
return -1;   // key not found
```


Java™

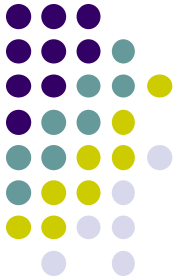
WIX1002
Fundamentals of Programming

Chapter 6
Java Methods

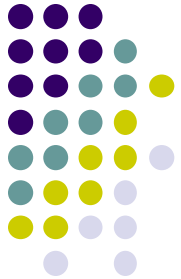


Contents

- Methods
- Static Method
- Reference Type



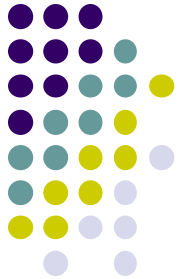
Methods



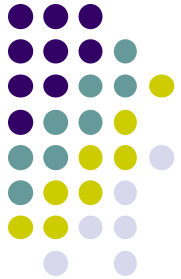
- Modules in Java are called methods and classes. Methods allow the programmer to modularize a program.
- To define the method

  accessSpecifier returnType
  methodName(parameterType parameterName, …) {
  // Method body
  }
- The accessSpecifier for methods is usually public.
- The returnType specified the type of variable return by the method. It can be the **primitive type or object type**. Use **void** if the method return nothing.

Methods



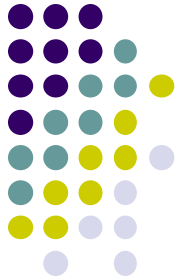
- The body of a method that returns a value must contain at least one **return statement**.
- A return statement always end a method invocation.
- To return a value
  - **return variableName;**
- To end a void method, the return statement without any expression can be used
  - **return;**



Methods

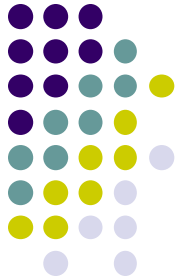
- Most methods have a list of parameters. The parameters provides the means for communicating information between methods via method calls.
- Methods can consists of one or more parameters or without parameter.
- A method is invoked by a method call. The method call specifies the **name of the method** and provides the correct parameters according to **parameter sequence and type**.
- When the method call completes, the method either returns a result to the calling method or simply returns control to the calling method.

Methods



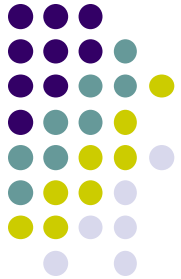
- A variable declared within a method is called a **local variable**.
- Each method can have the same variable name, however, these variable are different. The value is stored in different memory location.
- In Java, all the method invocation are **call-by-value**.
- **Call-by-value** means that when you invoke a method, a copy of the value of each actual parameter is passed to the method. Any changes to the copy inside the method will have **no effect on the actual parameter**.

Methods



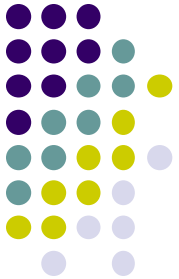
- Object and array are known as **reference type**. Java does not manipulate objects and arrays directly. Instead, it manipulates **references to objects and arrays**.
- Thus, when object and array are used as parameter in the method, **any changes** to the instance variable of the object or items of the array **will have effect on the actual parameter**.

Methods



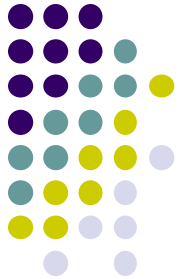
- Each method contains two types of comment namely the **precondition** and the **postcondition**.
- Precondition states what is assumed to be true when the method is called.
- Postcondition describes the effect of the method call. It describes the value returned by the method.

Static Method



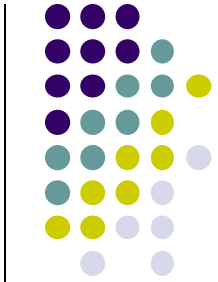
- Static method is the method that **do not require an calling object**.
- The definition of static method is inside the class definition.
- To define a static method
  - public static returnType methodname(parametertype parameter, ...)
- To invoke the a static method in the **same class**, just used the **methodname**.
- To invoke the a static method from **different class**, use **classname.methodname**.

Static Method

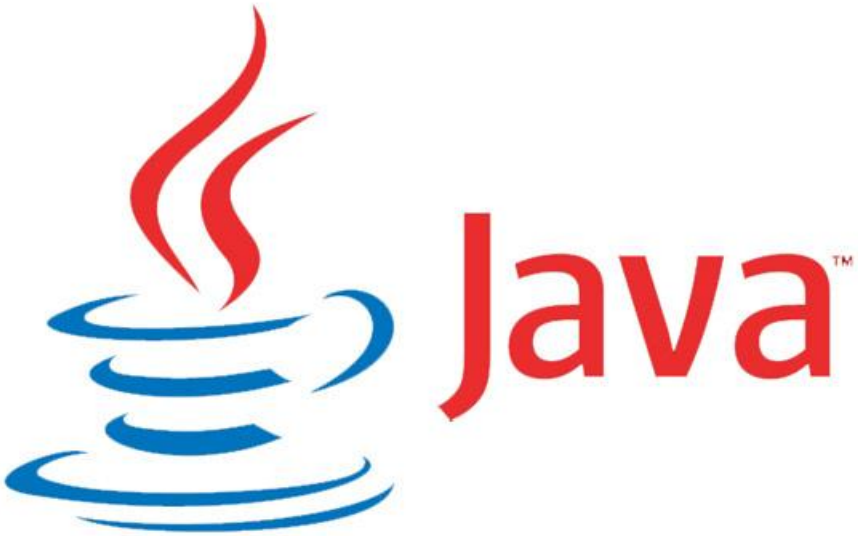


- The main method is the static method.
  - public static void main(String[] args)
- The main method is used to start the program. It contains a parameter which is an array of String.

Reference Type

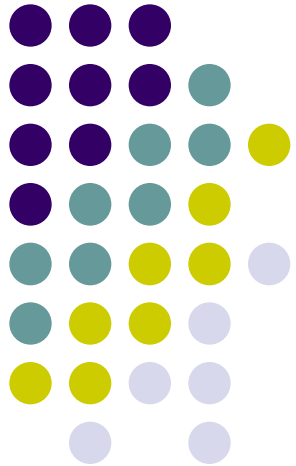


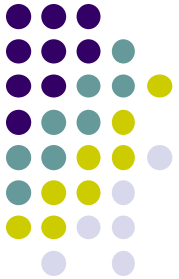
- Reference types is the variable that contains references.
- In Java, a variable of class type stores the reference of where the object is located in the memory.
- Thus, a method can change the values of the instance variables of an argument of a class type.
- The Java primitive types are not the reference type.
- The constant **null** can be assigned to a variable of class type. null is used to indicate that the class type variable has no real value.


Java™

WIX1002
Fundamentals of Programming

Chapter 7
File Input and Output

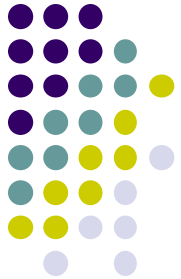




Contents

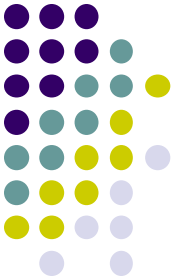
- Introduction
- Writing to Text File
- Reading from Text File
- File Class
- Writing to Binary File
- Reading from Binary File

Introduction



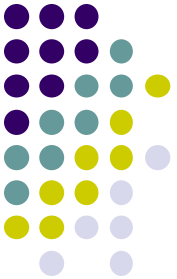
- Files are used for **permanent storage** of large amounts of data
- **Text file** is file that **contains sequence of characters**. It is sometimes called ASCII files because the data are encoded using **ASCII coding**.
- **Binary file** stores data in binary format. The data are stored in the **sequence of bytes**.
- A stream is a flow of data. If the data flows into the program, the stream is **input stream**. If the data flows out of the program, the stream is **output stream**.

Writing to Text File



- **PrintWriter** class is used to write data to a text file.
- PrintWriter streamObject = new PrintWriter(new FileOutputStream(FileName));
- Close the file after finish writing using streamObject**.close()** method.
- The **PrintWriter, FileOutputStream and IOException** class need to be loaded using the import statement.

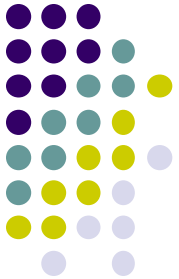
Writing to Text File



```
import java.io.PrintWriter;
import java.io.FileOutputStream;
import java.io.IOException;

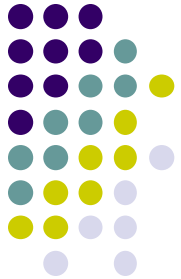
try {
   PrintWriter outputStream = new PrintWriter(new
    FileOutputStream("data.txt"));
   …
   outputStream.close();
} catch (IOException e) {
   System.out.println(“Problem with file output");
}
```

Writing to Text File



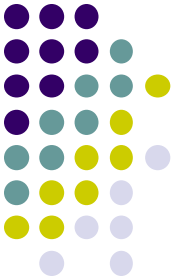
- After the outputStream has been declared, **print, println** and **printf** can be used to write data to the text file.
- To write to the file on a specified directory,
  - PrintWriter outputStream = new PrintWriter(new FileOutputStream(**"d:/sample/data.txt"**));
- To **append** to a text file
  - To write to the end of the file,
    - PrintWriter outputStream = new PrintWriter(new FileOutputStream("d:/sample/data.txt", **true**));

Reading from Text File



- Two most common stream classes used for reading text file are the **Scanner** class and **BufferReader** class.
- Scanner streamObject = new Scanner (new FileInputStream(FileName));
- Close the file after finish reading using streamObject**.close()** method.
- The **FileInputStream** and **FileNotFoundException** class need to be loaded using the import statement.

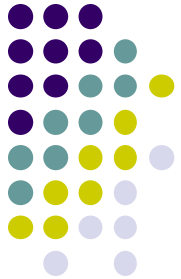
Reading from Text File



```
import java.util.Scanner;
import java.io.FileInputStream;
import java.io.FileNotFoundException;

try {
   Scanner inputStream = new Scanner(new
    FileInputStream("data.txt"));
   …
   inputStream.close();
} catch (FileNotFoundException e) {
   System.out.println("File was not found");
}
```

Reading from Text File



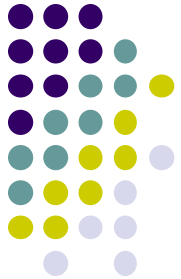
- After the inputStream has been declared, **nextInt, nextDouble, nextLine** can be used to read data from the text file.

  String input = inputStream.nextLine();

  int num1 = inputStream.nextInt();

  double num2 = inputStream.nextDouble();
- To check for the end of a text file
  - while (**inputStream.hasNextLine()**)
- To open file from a specified directory
  - Scanner inputStream = new Scanner(new FileInputStream(**"d:/sample/data.txt"**));

Reading from Text File

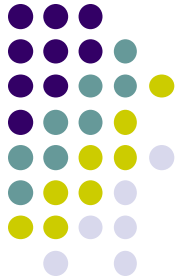


- BufferedReader class is another class that can read text from the text file.
- BufferedReader inputStream = new BufferedReader(new FileReader(FileName));
- Close the file after finish reading using streamObject**.close()** method.
- The **BufferedReader**, **FileReader** and **FileNotFoundException, IOException** class need to be loaded using the import statement.

Reading from Text File

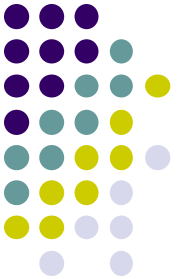
```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileNotFoundException;
import java.io.IOException;
try {
   BufferedReader inputStream = new BufferedReader (new
     FileReader("data.txt"));
   …
   inputStream.close();
} catch (FileNotFoundException e) {
   System.out.println("File was not found");
} catch (IOException e) {
   System.out.println(“Error reading from file");
}
```

Reading from Text File



- After the inputStream has been declared, **read** and **readLine** can be used to read data from the text file.

  String input = inputStream.readLine();
- To check for the end of a text file
  - while (**(input=inputStream.readLine()) != null**)
- To open file from a specified directory
  - BufferedReader inputStream = new BufferedReader(new FileReader("d:/sample/data.txt"));

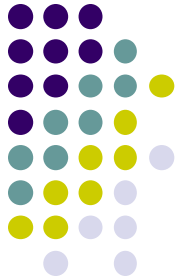


File Class

- File class contains methods that used to check the **properties of the file**.
- The file class is loaded using **import java.io.File;**

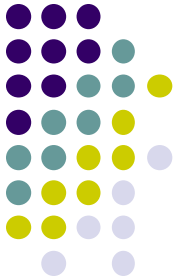
```
File fileObject = new File("data.txt");
if (fileObject.exists()) {
   System.out.println("The file is already exists");
   fileObject.renameTo("data1.txt");
}
if (fileObject.canRead())
   System.out.println("The file is readable");
if (fileObject.canWrite())
   System.out.println("The file is writable");
```

Writing to Binary File



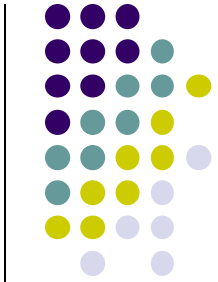
- **ObjectOutputStream** is the stream class that used to write data to a binary file.
- ObjectOutputStream streamObject = new ObjectOutputStream (new FileOutputStream(FileName));
- The **ObjectOutputStream, FileOutputStream and IOException** class need to be loaded using the import statement.
- The **writeInt, writeDouble, writeChar, writeBoolean** can be used to write the value of different variable type to the output stream. Use **writeUTF** to write String object to the output stream.
- Close the file after finish writing using streamObject**.close()** method.

Writing to Binary File



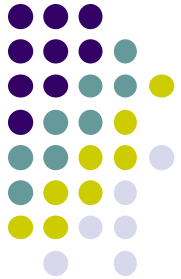
```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.FileOutputStream;
try {
   ObjectOutputStream outputStream = new
    ObjectOutputStream (new FileOutputStream("data.dat"));
   …
   outputStream.close();
} catch (IOException e) {
   System.out.println("Problem with file output.");
}
```

Reading from Binary File



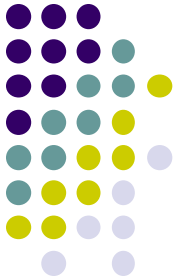
- **ObjectInputStream** is the stream class that used to read a binary file written using **ObjectOutputStream**
- ObjectInputStream streamObject = new ObjectInputStream (new FileInputStream(FileName));
- The **ObjectInputStream, FileInputStream and IOException, FileNotFoundException** class need to be loaded using the import statement.
- The **readInt, readDouble, readChar, readBoolean** can be used to read the value from the input stream. Use **readUTF** to read String object from the input stream.
- Close the file after finish writing using streamObject**.close()** method.

Reading from Binary File



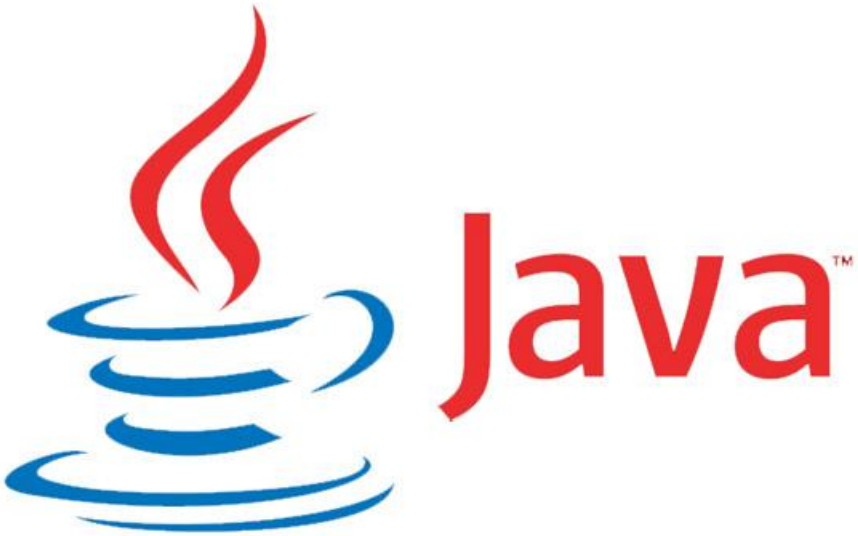
```
import java.io.IOException;
import java.io.FileNotFoundException;
import java.io.ObjectInputStream;
import java.io.FileOutputStream;
try {
   ObjectInputStream inputStream = new ObjectInputStream (new
     FileInputStream("data.dat"));
   …
   inputStream.close();
} catch (FileNotFoundException e) {
   System.out.println("File was not found");
} catch (IOException e) {
   System.out.println("Problem with file input.");
}
```

Reading from Binary File



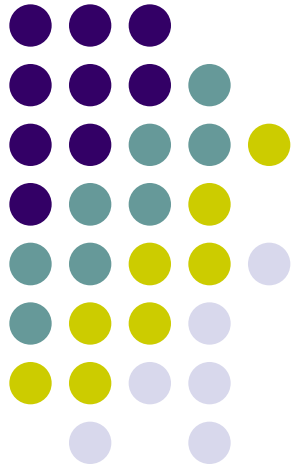
- To check for the end of a text file
  - Use **EOFException**

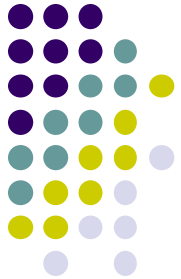
```
try {
   while(true) {
      number = inputStream.readInt();
   }
} catch (EOFException e) { }
```


Java™

WIX1002
Fundamentals of Programming

Chapter 8
Class

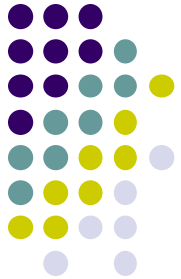




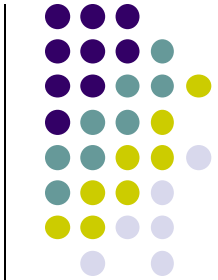
Contents

- Introduction
- Constructor
- Instance Variables
- Static Variable
- Class method
- Encapsulation
- Method Overloading
- Wrapper Class
- Package

Introduction



- Java is an Object-oriented programming language (OOP).
- OOP is a programming methodology that views a program as similarly consisting of **objects** that interact with each other by means of **actions**.
- An object is a software bundle of related **state and behavior**. Software objects are often used to model the real-world objects in everyday life.
- Objects of the same kind are in the same **class**. Example, a Perodua Myvi white car and a Perodua Myvi blue car are belongs to the same class.
- The actions of an object are called **methods**.



Introduction

- Java consists of objects from various classes interacting with one another.
- A class is a prototype from which objects are created. It models the state and behavior of a real-world object.
- A class consists of **constructor**, data items (**instance variables**) and **methods**.
- To define a class

```
public class className {
    // instance variables
    // constructors
    // methods
}
```

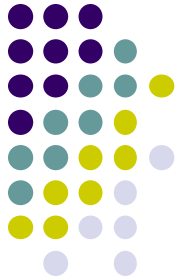
Introduction

- Once the class has been defined. The object of the class can be created in the tester program.
- To create the object of the class

  className obj = **new** className();

```
public class Test {
   public static void main(String[] args) {
        person p = new person();
        // ...
   }
}
```

Constructor



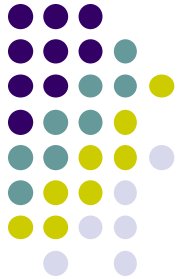
- Constructor is a special variety of method that used to initialize instance variable.
- Constructor must have the **same name as the class**.
- To define constructor

```
public className() {

}
public className(parameterType parameterName, …) {

}
```

Constructor

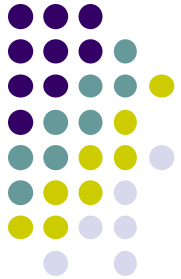


- Constructor without parameters is called no-argument constructor.
- Java will automatically creates a no-argument constructor if the class definition does not have a constructor.

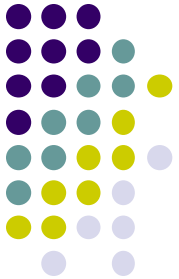
Constructor

```
public person() {
        name = null;
        age = 0;
        gender='\0';
}
person p = new person();

public person(String n, int a, char g) {
        name = n;
        age = a;
        gender=g;
}
person p = new person("John", 30, 'M');
```



Constructor

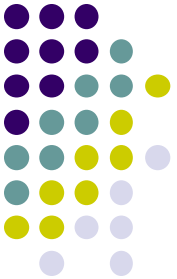


- A copy constructor is a constructor with one parameter of the same type as the class. It creates an object that is separate, independent object with the instance variables are **exact copy of an argument object**.

```
public person(person p) {
   if (p==null)
      System.exit(0);

   name = p.getName();
   age = p.getAge();
   gender= p.getGender();
}
```

Instance Variables



- An object used instance variables to store its state.
- To declare instance variables

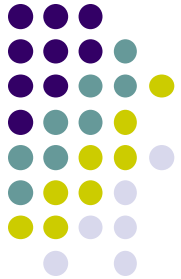
  accessSpecifier variableType variableName;

  private String name;

  private int age;

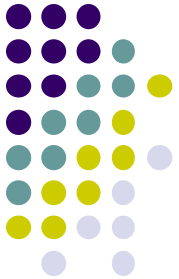
  private char gender;
- The accessSpecifier for instance variable is always private.
- Access specifiers consists of **private, protected and public**.

Instance Variables



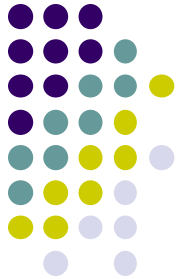
- **public** means that there are no restrictions on where the instance variable can be used.
- **private** means that the instance variable is accessible only by methods of the same class.
- **protected** means that the instance variable is accessible by methods of the same class, subclass or class in the same package.
- A **package** is a namespace for organizing classes and interfaces in a **logical manner**. It makes large software projects easier to manage.

Instance Variables



Specifier	class	subclass	package	world
private	Y			
protected	Y	Y	Y	
public	Y	Y	Y	Y

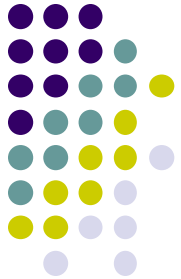
Instance Variables



- In Java method definition, the **this** keyword can be used as the **calling object**.
- The this parameter refer to the **reference to the current object**, the object whose method or constructor is being called.

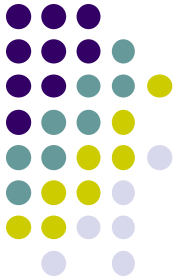
```
public void setName(String name) {
   this.name = name;
}
```

Static variable



- A static variable is a variable that belongs to the class. There is only **one copies of the static variable for all the objects**.
- A static variable can be used by **objects to communicate** between the objects.
- To declare a static variable
  - private static int counter;
- To declare a static constants
  - private static final int MAX = 100;
- **A static method can access the static variables but can't access the instance variables**.

Class Method



- **Accessor method** are used to **get the value** of instances variable

  public String getName() }

  return name;

  }
- **Mutator method** are used to **set or modify** the instances variable.

  public void setName(String n) {

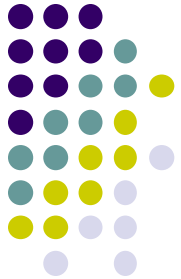
  name = n;

  }

Class Method

```
public class Test {
   public static void main(String[] args) {
        person p = new person();
        p.setName("Alvin");
        System.out.println("The name is " + p.getName());
   }
}
```

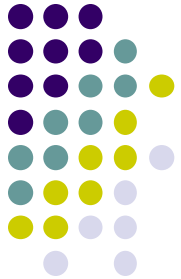
Class Method



- In Java, the class definition usually contains the **equals method** and the **toString method**.
- An equals method compare the calling object to another object and return true if both objects are equal.

```
public boolean equals(Person p) {
   if (name.equals(p.getName())
      return true;
   else
      return false;
}
```

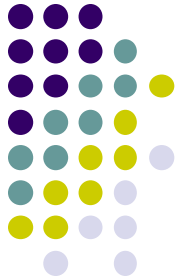
Class Method



- The toString method return a string representation of the data in the calling object.

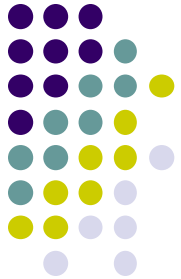
```
public String toString() {
   return "Name is " + this.name;
}
```

Encapsulation



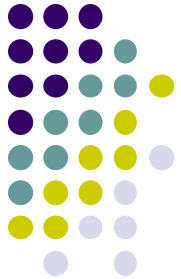
- **Encapsulation** means that the data and actions are combined into a single item which is a Class.
- It is used to group software into a unit that is easy to use with the simple interface.
- By defining the **Application Programming Interface (API)**, programmer can use it for any implementation.

Method Overloading



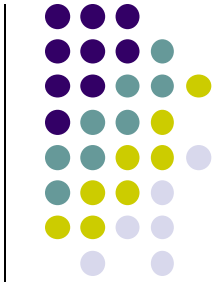
- **Method overloading** allows one class to have more than one definitions of a single method name.
- When a method is overloading, the definition must have different signatures either in **different number of parameters** or **some parameter position must be of differing types**.
- Java does not support **operator overloading**.

Method Overloading



```
public void setValue(String n) {
   name = n;
}
public void setValue(String n, int a) {
   name = n;
   age = a;
}
public void setValue(String n, int a, char g) {
   name = n;
   age = a;
   gender = g
}
```

Wrapper Class



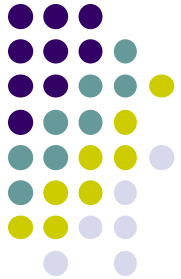
- Every Java primitive type has a corresponding wrapper class.
- The wrapper class contains useful predefined constant and static methods.
- Create object of a wrapper class

  Integer num = new Integer(100);

  Integer num1 = 50;

  Double price = 10.59;

  Character grade = 'C';



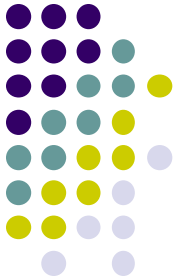
Wrapper Class

- Each wrapper class contains the instance variable for maximum and minimum value

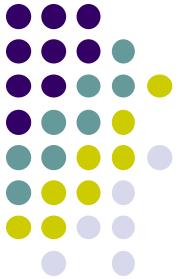
  Integer.MAX_VALUE

  Integer.MIN_VALUE
- The wrapper class also contains the method to convert String value to their type or from their type to String value.
- Integer.parseInt(String) // convert String to integer
- Double.parseDouble(String) // convert String to double
- Integer.toString(intValue) // convert integer to String

Wrapper Class



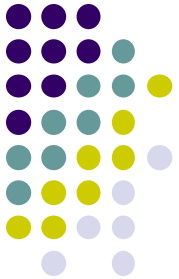
- Character Wrapper Class
  - .toUpperCase(charVariable);
  - .toLowerCase(charVariable);
  - .isUpperCase(charVariable);
  - .isLowerCase(charVariable);
  - .isWhitespace(charVariable);
  - .isLetter(charVariable);
  - .isDigit(charVariable);
  - .isLetterOrDigit(charVariable);



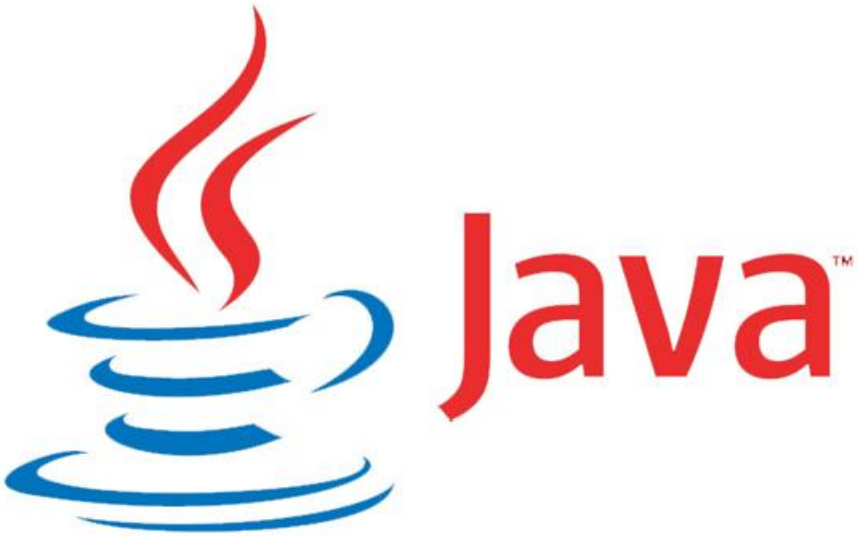
Package

- A package is a collection of classes that have been grouped together into a directory.
- Each package is given a package name. The package name is the path name to the directory containing the classes in the package.
- In Java, the CLASSPATH variable is used to locate Java packages. The new package will store in the directory **CLASSPATH/packName** folder.
- Each file in the same package must contain the same package name.
  - package packageName;

Package

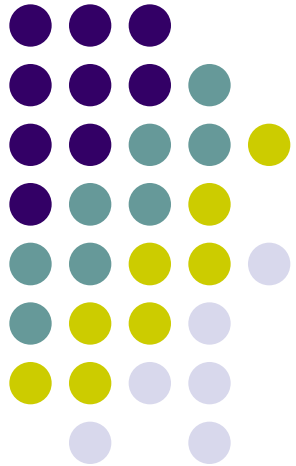


- Whenever a package is import, it does not import the subdirectory packages.
- All the classes in the current directory are called default package. The current directory of the default package is the CLASSPATH
- When compiling a Java file, Java use the default CLASSPATH. In order to use different CLASSPATH for compiling a Java file
  - javac **-classpath .;newClassPathDirectory** fileName.java
  - **. refer to current directory**


Java™

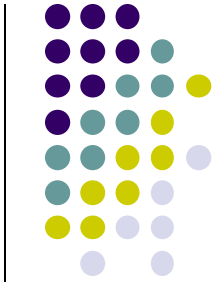
WIX1002
Fundamentals of Programming

Chapter 9
Inheritance

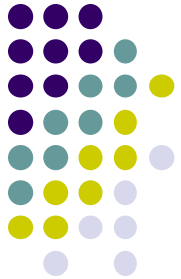


Contents

- Introduction
- Overriding Method
- Protected
- Package Access
- Object Class

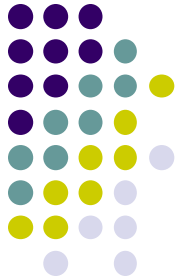


Introduction



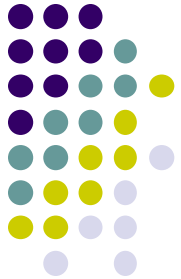
- One of the main techniques of OOP is known as inheritance. Advantage of inheritance is **code reuse**.
- **Inheritance** is the process by which a new class known as a **derived class** is created from another class called the **base class**.
- Inheritance is the mechanism for extending existing classes by adding methods and fields.
- The base class sometimes refer to **superclass** and the more specialized class that inherits from the superclass is called the **subclass**.
- A subclass automatically has all the instance variables, static variables and the public methods of superclass.

Introduction



- However, a subclass has **no access to the private fields** of its superclass.
- Sometimes the base class is called the **parent class** and the derived class is called the **child class**.
- If A is the parent class of another class, it is often called **ancestor class**. If A is an ancestor of B, B is often called a **descendent** of class A.
- **super()** invoke the no-argument constructor of superclass.
- super can also be used to call the method of a superclass.

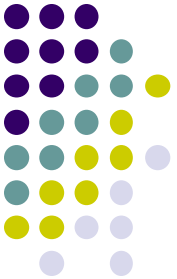
Introduction



```
public class subClassName extends superClassName {
   instance variables
   methods
}

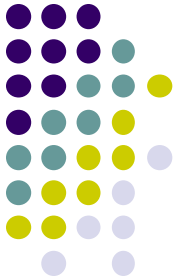
public class HourlyEmployee extends Employee {
    public HourlyEmployee() {
       super();
       super.superClassMethod();
    }
}
```

Overriding Method



- A derived class inherits method that belong to base class. If the base class requires different definition for an inherit method, the method can be redefined or override.
- However, if the base class method consists of **final** modifier, the method can not be overridden.
- As a general rule, an override method cannot change the return type of the method definition. However, if the returned type is a class type, the returned type can be changed to the **descendant class type**.
- An override method can change the **private method** in base class **to public method** in derived class. However, it can't change the public method to private method.

Overriding Method



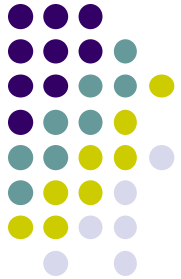
```
public class BaseClass {
   ...
   public Employee getValue() {

   }
}

public class DerivedClass extends BaseClass {
   ...
   public HourlyEmployee getValue() {

   }
}
```

Overriding Method



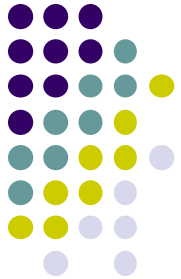
```
public class BaseClass {
   ...
   private void setValue(int a) {

   }
}

public class DerivedClass extends BaseClass {
   ...
   public void setValue(int a) {

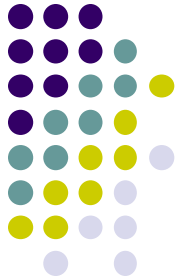
   }
}
```

Protected



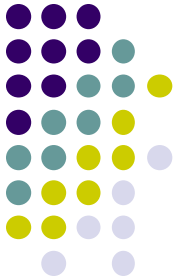
- An object of an derived class has the type of the derived class as well as the type of the base class and every one of its ancestor classes.
- However, a derived class **can't access directly** to the **private instance variables** of the base class.
- The **protected** modifier can be used to allow the access for its own class and the derived class.

Package Access



- A package is a namespace for organizing classes and interfaces in a logical manner. Java packages can be stored in compressed files called **JAR** files.
- If no modifiers like public, protected or private for instance variables or methods, the instance variables or methods will have **package access**.
- Package access is the default access where it can be accessed by name inside the definition of any class in the same package but not outside the package.

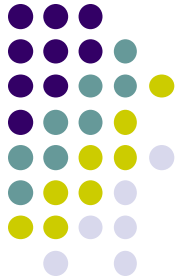
Object Class



- In Java, every class is a derived class of the class Object.
- Every class inherit the **getClass** method from the class Object. The getClass method can be used to check the class of an object. The getClass method can't be overridden.

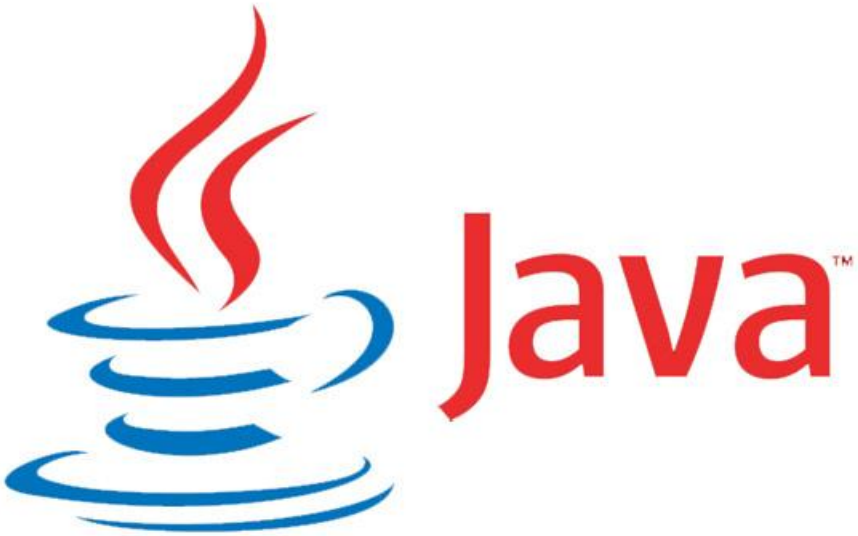
```
if (obj1.getClass() == obj2.getClass())
   System.out.println("The objects are belongs to same
    class");
```

Object Class



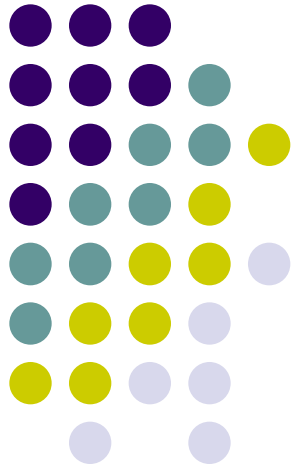
- The **instanceof** operator checks if an object if of type given as its second argument.
- The instanceof will return true if the object is of the type of any descendent class.

```
if (obj1 instanceof ClassName)
   System.out.println("The object is one of the type of any
    descendent class of ClassName");
```


Java™

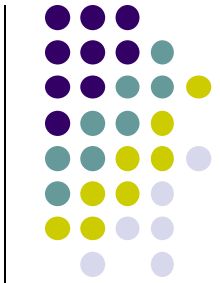
WIX1002
Fundamentals of Programming

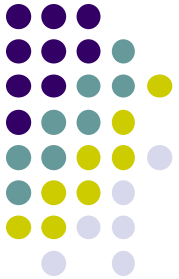
Chapter 10
Polymorphism



Contents

- Polymorphism
- clone Method
- Abstract Classes
- Interface
- Comparable Interface
- Inner Class

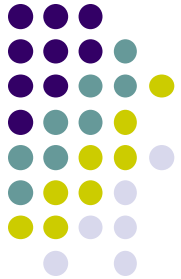




Polymorphism

- **Polymorphism** refers to the ability to associate many meanings to one method name using late binding or dynamic binding
- **Binding** refers to the process of associating a method definition with a method invocation.
- **Early Binding** associate a method definition with the method invocation when the code is compiled.
- **Late binding / Dynamic Binding** associate a method definition with the method invocation when the method is invoked (**at run time**)
- Java does not use late binding with private method, methods with final modifier and static method.

Polymorphism



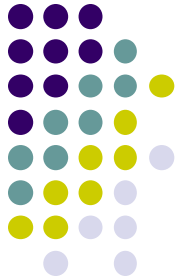
- An object of a derived class has the type of its base class.
- Assigning an object of a derived class to a variable of base classis often called **upcasting**.

  BaseClass a = new BaseClass;

  DerivedClass b = new DerivedClass;

  a = b;

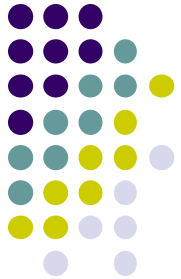
Polymorphism



- Downcasting assigning an object of a base class to a derived class.
- To test whether the downcasting is legitimate

```
DerivedClass obj = new DerivedClass();
if (obj1 instanceof DerivedClass) {
   obj = (DerivedClass) obj1
```

clone Method

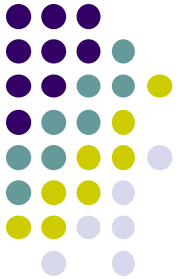


- Every object inherits a method named clone from the class Object.
- The clone method **return a copy of the calling object**.

```
public ClassName clone() {
   return new ClassName(this);
}

public Sale doCopy(Sale a) {
   Sale b;
   b = a.clone();
}
```

Abstract Classes

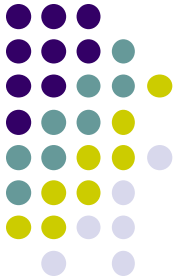


- An abstract class is a class that has **some methods without complete definitions**. A class with no abstract methods is called a concrete class.
  - public abstract class className
- Object can't be created using abstract class constructor.
- An **abstract method** serves as a placeholder for a method that will be fully defined in a descendent class. It has **no method body**.
- An abstract method can't be private
  - public abstract returnType methodName(parameterType parameterName, …);

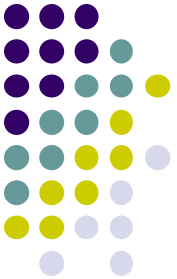
Abstract Classes

```
public abstract class Employee {
   public abstract double getPay();
   ...
}

Employee a = new Employee()
// can't create object for an abstract class
```



Interface



- A Java interface specifies a set of methods that any class that implements the interface must have.
- An **interface** is a type that groups together a number of different classes that all include method definitions for a common set of method headings.
  - public interface interfaceName
- An interface has **no instance variable** and **no method definitions**.
- An interface can contain defined **constants** as well as **method headings**. When a method implements the interface, it automatically gets the defined constant.

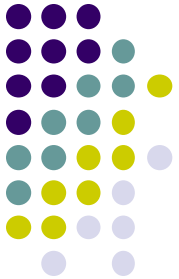
Interface

```
public interface interfaceName {

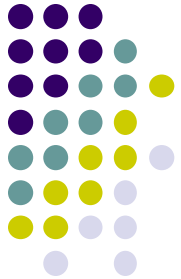
    public static final int MAX = 100;

    public returnType methodName(parameterType
     parameterName, ...);

}
```

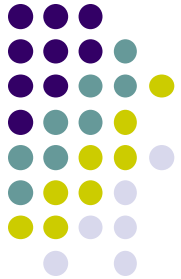


Interface



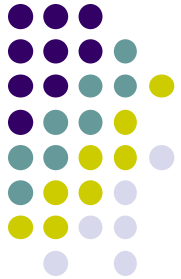
- The class must **implement all the method headings** listed in the definitions of the interfaces.
  - public class className **implements interfaceName, …**
- When a class implements an interface, it must make all the methods in the interface public.
- In Java, a derived class can have only **one base class**. However, a class can implement any number of interfaces.

Comparable Interface



- Comparable interface has one method
  - **public int compareTo(Object other);**
  - The method return negative if calling object comes before the parameter object
  - The method return positive if calling object comes after the parameter object
  - The method return zero if calling object equals to the parameter object

Inner Class



- Inner class are classes defined within other classes.
- Inner and outer classes have access to each other's private members

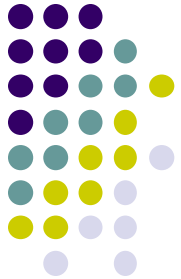
```
public class ClassName {

   private class ClassName {

   }

}
```

Inner Class



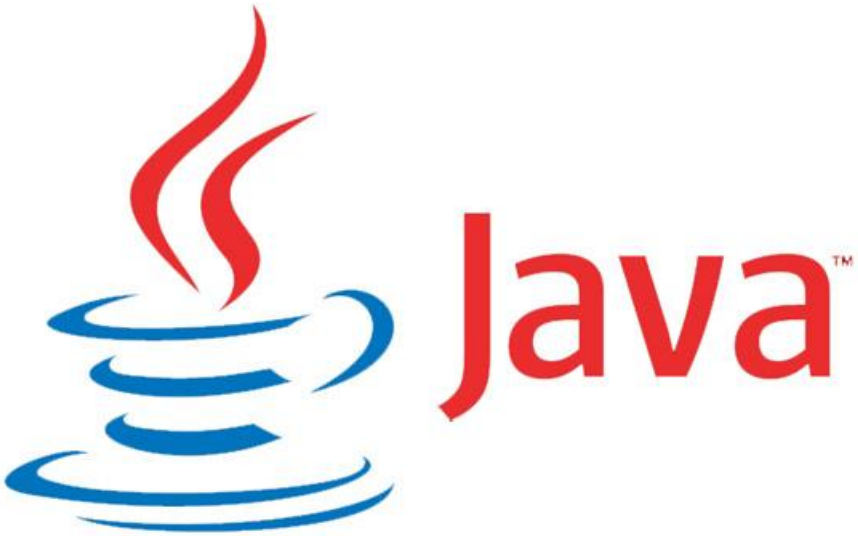
- A static Inner class **can have nonstatic instance variables and methods** but an object of static inner class has no connection to an object of the outer class.

```
public class ClassName {

   private static class ClassName {

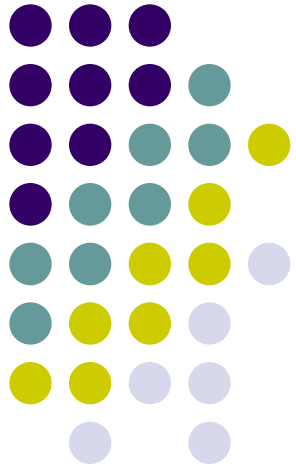
   }

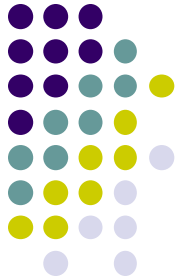
}
```


Java™

WIX1002
Fundamentals of Programming

Chapter 11
Exception Handling

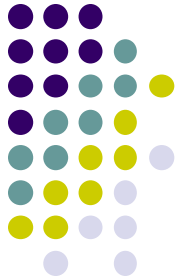




Contents

- Exception Handling
- try-catch
- Exception Class
- Exception in Method
- finally

Exception Handling

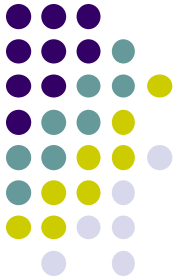


- **Exception handling** is a very important aspect of writing robust software. When an error occurs in a Java program it usually results in an exception being thrown.
- An exception represents an error condition that can occur during the normal course of program execution.
- When an exception occurs, an exception is thrown.
- By using exception handling, the exception is caught and processed.
- **try-catch** statement is used for exception handling.

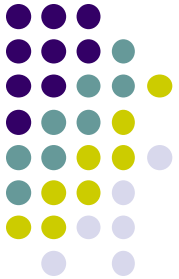
try-catch

```
try {
   // try block
} catch (Exception e) {
   // catch block
}
```

```
try {
   throw new Exception("Exception Description");
} catch (Exception e) {
   System.out.println(e.getMessage());
}
```



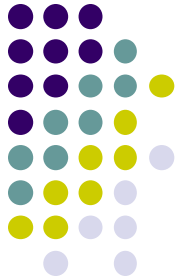
try-catch



- Multiple catch blocks

```
try {
   // try block
} catch (ExceptionOne e) {
   // catch block
} catch (ExceptionTwo e) {
   // catch block
}
```

try-catch



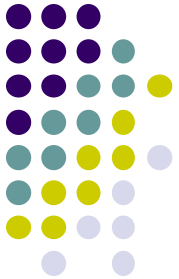
- Nested catch blocks

```
try {
   // try block

   try {
      // try block
   } catch (ExceptionOne e) {
      // catch block
   }

} catch (ExceptionOne e) {
   // catch block
}
```

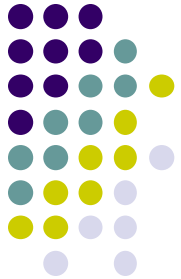
Exception Class



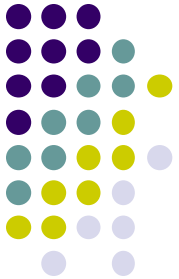
- Java includes some predefined exception classes.
- Some predefined exceptions are
  - IOException
  - NoSuchMethodException
  - FileNotFoundException
  - NumberFormatException
  - DivisionByZeroException
  - ArrayIndexOutOfBoundsException
- The new exception class can be defined. An exception class can be a derived class of any exception class.

Exception Class

```
public class exceptionClassName extends Exception {
   public exceptionClassName() {
      super("Error Message");
   }
   public exceptionClassName(String s) {
      super(S);
   }
}
```



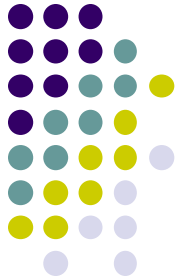
Exception in Method



- Sometimes an exception can be thrown in a method without catching it in the same method.
- The method will stop if the exception is thrown.
  - public returnType methodName(parameterType parameterName, ..) **throws ExceptionName, …**

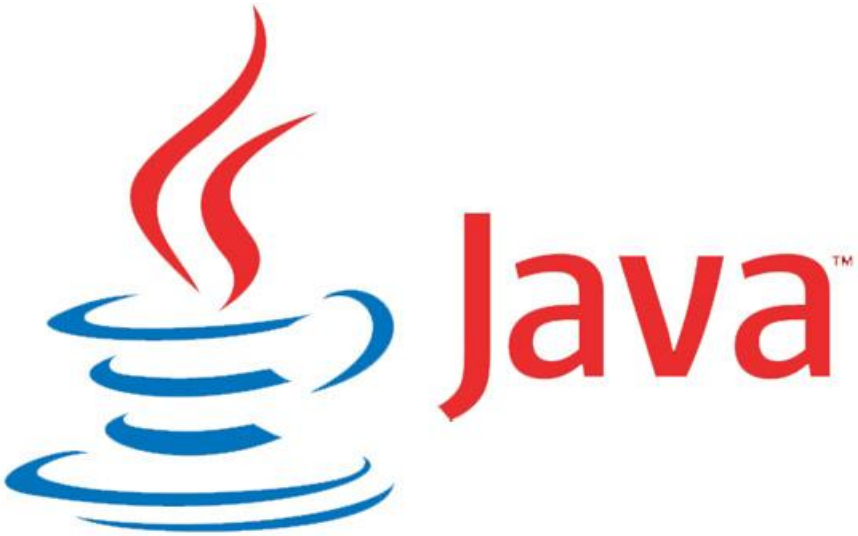
```
try {
   // try block
   methodName();
} catch (Exception e) {
   // catch block
}
```

finally



- The finally block contains code to be executed whether or not an exception is thrown in a try block.
- The finally always execute in the try block.

```
try {
   // try block
} catch (ExceptionOne e) {
   // catch block
} finally {
   // Code to be executed whether or not an exception is
    thrown
}
```


Java™